

LAPORAN TUGAS BESAR 3
IF2211 Strategi Algoritma

**Implementasi Pattern Matching pada Chromium Browser Extension
untuk Deteksi Judi Online**



Disusun oleh Kelompok: SweetBonanza

13524036 Edward David Rumahorbo

13524066 Nathanael Gunawan

13524102 Manuel Thimoty Silalahi

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

Daftar Isi

1. Deskripsi Tugas.....	3
1.1. Latar Belakang.....	3
1.2. Tujuan.....	4
1.3. Ruang Lingkup.....	4
2. Landasan Teori.....	5
2.1. Chromium Browser Extension.....	5
2.2. Document Object Model (DOM).....	6
2.3. Knuth-Morris-Pratt.....	6
2.4. Boyer-Moore.....	7
2.5. Regular Expression (RegEx).....	8
2.6. Weighted Levenshtein Distance.....	9
2.7. Optical Character Recognition (OCR).....	11
2.8. Aho-Corasick.....	11
2.9. Rabin-Karp.....	12
3. Analisis Pemecahan Masalah.....	13
3.1. Alur Kerja Sistem.....	13
3.2. Preprocessing dan Normalisasi Teks.....	15
3.3. Penerapan KMP dan Boyer-Moore.....	16
3.4. Penerapan RegEx.....	17
3.5. Penerapan Weighted Levenshtein Distance.....	18
3.6. Pengolahan DOM.....	19
3.7. Statistik pada Popup Extension.....	20
3.8. Fitur Bonus.....	21
3.8.1. Blur Teks.....	21
3.8.2. OCR Gambar.....	21
3.8.3. Aho-Corasick dan Rabin-Karp.....	22
4. Implementasi dan Pengujian.....	24
4.1. Lingkungan Pengembangan.....	24
4.2. Struktur Program.....	25
4.3. Implementasi Algoritma.....	27
4.3.1. Knuth-Morris-Pratt'.....	27
4.3.2. Boyer-Moore.....	29
4.3.3. Regular Expression.....	31
4.3.4. Weighted Levenshtein Distance.....	32
4.3.5. Aho-Corasick.....	34
4.3.6. Rabin-Karp.....	38

4.4. Implementasi Browser Extension.....	40
4.4.1. Manifest.json.....	40
4.4.2. Background Service Worker.....	40
4.4.3. Content Script & Pemindaian DOM.....	41
4.4.4. Highlight dan Tooltip.....	41
4.4.5. MutationObserver dan Rescan.....	42
4.4.6. OCR Gambar.....	42
4.4.7. Penyimpanan Statistik.....	42
4.4.8. Settings Store dan Blur Teks.....	43
4.4.9. Popup.....	43
4.5. Pengujian.....	43
4.5.1. Hasil Pengujian.....	43
4.5.2. Hasil Performa Algoritma.....	52
4.5.3. Analisis Hasil Pengujian.....	54
5. Kesimpulan dan Saran.....	56
5.1. Kesimpulan.....	56
5.2. Saran.....	57
LAMPIRAN.....	58
Link Repository GitHub.....	58
Tabel Checklist.....	58
Pernyataan Tidak Melakukan Kecurangan.....	59
Tabel Pembagian Tugas.....	59
Referensi.....	60

1. Deskripsi Tugas

1.1. Latar Belakang

Perkembangan internet telah mempermudah masyarakat dalam memperoleh informasi, berkomunikasi, dan mengakses berbagai layanan secara digital. Namun, kemudahan ini juga membuka peluang untuk penyebaran konten yang tidak diinginkan, contohnya adalah judi online. Konten tersebut dapat ditemukan pada berbagai bagian halaman web, seperti judul, paragraf, link, iklan, maupun gambar. Kemunculannya tidak hanya mengganggu pengguna yang ingin menggunakan layanan tertentu, tetapi juga meningkatkan risiko paparan ke situs perjudian ilegal.

Upaya yang dilakukan untuk mendeteksi konten judi online tidak selalu dapat dilakukan hanya dengan mencari kata tertentu secara langsung. Nyatanya, promosi judi online sering menggunakan variasi penulisan untuk menghindari deteksi sederhana, misalnya seperti menggunakan pola gabungan kata dan angka, seperti GACOR99, MAXWIN234, dan SLOT88. Selain itu, terdapat pula penggunaan karakter pengganti yang secara visual menyerupai karakter aslinya, seperti angka 0 untuk menggantikan huruf O, angka 1 untuk menggantikan huruf I, atau karakter Unicode tertentu yang menyerupai huruf alfabet. Sebagai contoh, teks HOKI88 dapat ditulis sebagai H0KI88, sedangkan Gacor999 dapat disamarkan menjadi Gȃcor999.

Permasalahan ini termasuk ke dalam persoalan pattern matching pada teks. Algoritma Knuth-Morris-Pratt dan Boyer-Moore dapat digunakan untuk mencari kemunculan kata kunci secara tepat (exact matching) berdasarkan daftar kata yang telah ditentukan. Regular Expression (Regex) dapat digunakan sebagai metode tambahan untuk mengenali pola kata yang diikuti oleh angka. Sementara itu, Weighted Levenshtein Distance dapat digunakan untuk mendeteksi kata-kata yang memiliki tingkat kemiripan tinggi dengan kata kunci, termasuk variasi yang dihasilkan dengan mengganti karakter yang serupa secara visual.

Berdasarkan permasalahan tersebut, kami telah mengembangkan sebuah Chromium browser extension yang bisa memindai konten pada halaman web secara otomatis. Extension ini mendeteksi teks yang terindikasi mengandung kata-kata yang berhubungan dengan judi online, memberikan penanda visual pada elemen yang terdeteksi, menampilkan informasi hasil pencocokan melalui tooltip, serta menyajikan statistik pemindaian secara real-time melalui halaman popup. Selain itu, extension juga dilengkapi dengan fitur tambahan berupa penyensoran teks melalui efek blur dan pemanfaatan Optical Character Recognition (OCR) untuk mendeteksi teks yang disisipkan pada gambar.

1.2. Tujuan

Tujuan pengembangan *Chromium browser extension* untuk deteksi konten judi online ini adalah sebagai berikut:

1. Mengembangkan Chromium browser extension berbasis TypeScript yang dapat mendeteksi konten yang berhubungan dengan judi online pada halaman web.
2. Mengimplementasikan algoritma pattern matching, yaitu Knuth-Morris-Pratt, Boyer-Moore, dan Regular Expression, untuk mendeteksi kata kunci maupun pola teks tertentu seperti gabungan kata dan angka.
3. Menerapkan Weighted Levenshtein Distance untuk mendeteksi variasi kata kunci yang disamarkan melalui perubahan karakter, kesalahan penulisan, atau penggunaan karakter yang serupa secara visual.
4. Menyediakan fitur berupa highlight pada teks yang terdeteksi, tooltip informasi hasil deteksi, rescan halaman, dan statistik real-time pada popup extension.
5. Mengimplementasikan fitur tambahan berupa blur teks, OCR pada gambar, serta algoritma bonus apabila tersedia, untuk meningkatkan kemampuan deteksi terhadap berbagai bentuk penyamaran konten.

1.3. Ruang Lingkup

Ruang lingkup pengembangan extension ini mencakup hal-hal berikut:

- **Input:**
 - Konten teks pada halaman web.
 - Elemen gambar pada halaman web.
 - Daftar kata kunci dari berkas keywords.txt.
 - Pengaturan blur teks, OCR, dan perintah rescan dari popup extension.
- **Process yang dilakukan:**
 - Pencarian exact match menggunakan Knuth-Morris-Pratt dan Boyer-Moore.
 - Pendeteksian pola <kata><angka> menggunakan Regular Expression.
 - Pencarian fuzzy match menggunakan Weighted Levenshtein Distance.
 - Pemindaian teks pada gambar menggunakan OCR.

- Penandaan elemen DOM yang terdeteksi.
- **Output:**
 - Highlight pada teks yang mengandung indikasi judi online.
 - Tooltip berisi keyword, algoritma, jumlah kemunculan, dan waktu eksekusi.
 - Statistik real-time pada popup extension.
 - Efek blur pada teks atau gambar yang terdeteksi apabila fitur diaktifkan.
- **Batasan:**
 - Extension dijalankan pada browser berbasis Chromium.
 - KMP dan Boyer-Moore diimplementasikan tanpa fungsi pencarian bawaan atau library string matching eksternal.
 - Akurasi fuzzy matching bergantung pada threshold yang digunakan.
 - Akurasi OCR dipengaruhi oleh kualitas dan keterbacaan gambar.

2. Landasan Teori

2.1. Chromium Browser Extension

Chromium Browser Extension adalah aplikasi perangkat lunak kecil berbasis web (HTML, CSS, dan JavaScript) yang dipasang untuk memodifikasi dan meningkatkan fungsionalitas peramban berbasis Chromium. Fondasi utama dari setiap ekstensi adalah berkas `manifest.json`, yang bertindak sebagai berkas manifest konfigurasi sentral. Berkas ini mendefinisikan metadata ekstensi, hak akses keamanan (permissions), ikon, pustaka eksternal, serta mendaftarkan skrip-skrip aktif yang akan dijalankan oleh ekstensi untuk berinteraksi dengan browser dan halaman web.

Ekstensi Chromium membagi proses kerjanya ke dalam beberapa komponen terpisah, di antaranya adalah skrip konten (*content script*) dan pekerja latar belakang (*background service worker*). *Content script* adalah berkas JavaScript yang disuntikkan secara dinamis ke dalam halaman web aktif untuk membaca, memodifikasi, dan berinteraksi langsung dengan konten halaman tersebut. Sementara itu, *background service worker* berjalan secara asinkron di latar belakang peramban untuk mengelola daur hidup ekstensi, mendengarkan peristiwa browser, dan menangani tugas komputasi berat (seperti memproses Web Workers Tesseract.js pada OCR) tanpa memiliki akses langsung terhadap struktur visual halaman web.

Untuk mendukung interaksi dengan pengguna dan penyimpanan data, ekstensi menyediakan antarmuka popup extension yang berupa halaman HTML/CSS kustom yang muncul saat ikon ekstensi diklik, serta sistem penyimpanan lokal

chrome.storage.local. API penyimpanan lokal ini digunakan untuk menyimpan hasil pemindaian halaman dan konfigurasi secara asinkron dan persisten. Karena setiap komponen (skrip konten, popup, dan service worker) berjalan pada lingkungan isolasi memori yang berbeda, komunikasi antar komponen diwujudkan melalui mekanisme pertukaran pesan (message passing) menggunakan API pesan bawaan Chrome untuk mengirim dan menerima instruksi secara aman.

2.2. Document Object Model (DOM)

Document Object Model, atau disingkat DOM, adalah antarmuka pemrograman (Application Programming Interface) standar untuk dokumen HTML dan XML yang merepresentasikan seluruh struktur halaman web ke dalam bentuk pohon node (node tree). Melalui DOM, peramban dapat memetakan dokumen teks menjadi objek-objek logis yang terstruktur secara hierarkis, di mana setiap objek melambangkan bagian dari halaman web tersebut. Struktur pohon ini memungkinkan bahasa pemrograman seperti JavaScript untuk mengakses, memanipulasi, mengubah gaya, serta memperbarui konten dan struktur halaman secara dinamis setelah dokumen selesai dimuat oleh browser.

Di dalam hierarki pohon DOM, simpul-simpul penyusunnya dibedakan menjadi beberapa tipe node, di mana simpul elemen (element node) dan simpul teks (text node) menjadi komponen yang paling mendasar. Element node merepresentasikan tag-tag HTML (seperti <div>, <p>, atau), sedangkan text node merupakan simpul daun terendah yang menyimpan konten teks aktual yang berada di dalam tag HTML tersebut. Proses pemindaian konten teks dilakukan dengan melakukan penelusuran (traversal) pohon DOM untuk mengumpulkan seluruh text node yang terlihat secara visual menggunakan objek penelusur seperti TreeWalker, sambil mengabaikan simpul elemen yang tidak berisi informasi teks bagi pengguna.

Setelah teks judi online terdeteksi oleh suite algoritma, program melakukan modifikasi DOM dengan membungkus bagian teks yang cocok menggunakan elemen highlight (seperti tag <mark> atau dengan kelas CSS kustom). Pembungkusan ini disuntikkan secara aman pada posisi indeks yang tepat di dalam text node asli untuk memberikan penanda visual berupa warna stabilo atau efek buram (blur) tanpa merusak struktur atau tata letak (layout) halaman web. Pada saat pengguna memicu pemindaian ulang (rescanning), program wajib menjalankan fungsi pembersihan dengan melepas elemen pembungkus highlight lama dan mengembalikan struktur text node ke kondisi semula sebelum proses pemindaian baru dimulai guna mencegah penumpukan elemen dan kebocoran memori.

2.3. Knuth-Morris-Pratt

Algoritma Knuth-Morris-Pratt (KMP) merupakan algoritma pencocokan string efisien yang membandingkan karakter pola (*pattern*) dengan teks dari arah kiri ke kanan. Keunggulan utama algoritma ini dibandingkan metode konvensional (*Naive Algorithm*) terletak pada kemampuannya untuk mengeliminasi pemindaian ulang pada karakter teks yang sudah dipastikan cocok (*no backtracking*). Konsep ini diwujudkan dengan memanfaatkan sifat pengulangan substring di dalam pola itu sendiri melalui definisi prefix (awalan) dan suffix (akhiran). Dalam konteks ini, prefix sejati (*proper prefix*) didefinisikan sebagai potongan awal string yang tidak menyertakan keseluruhan string tersebut, sedangkan suffix merupakan potongan akhir string. Melalui analisis hubungan kedua elemen tersebut, algoritma KMP dapat menentukan langkah pergeseran pola secara dinamis ketika terjadi ketidakcocokan karakter (*mismatch*).

Sebelum proses pencarian dilakukan, algoritma KMP melakukan tahap pra-pemrosesan (*preprocessing*) terhadap pola untuk membangun sebuah fungsi batas (*border function*) yang direpresentasikan dalam tabel Longest Proper Prefix yang juga adalah *Suffix* (LPS Table) atau sering disebut *failure function*. Tabel LPS ini memiliki ukuran yang sama dengan panjang pola, di mana setiap indeks $lps[i]$ menyimpan nilai integer yang menyatakan panjang proper prefix terpanjang dari substring pola $pat[0..i]$ yang sekaligus bertindak sebagai *suffix* dari substring tersebut. Ketika proses pencocokan karakter di lapangan mengalami kegagalan pada indeks pola tertentu, program tidak akan mengulang perbandingan dari awal teks. Pointer pola akan secara otomatis digeser mundur ke indeks yang diarahkan oleh tabel LPS sebelumnya, sementara pointer teks tetap dapat melanjutkan pemindaian ke depan secara konsisten.

Dalam implementasi praktisnya, efisiensi waktu eksekusi algoritma diukur secara akurat dengan menyertakan mekanisme penghitungan jumlah perbandingan karakter (*comparison counting*). Variabel penghitung ini akan bertambah secara inkremental setiap kali terjadi operasi komparasi karakter, baik saat pembangunan tabel LPS maupun selama proses pemindaian teks berlangsung. Berkat eliminasi langkah mundur pada teks, algoritma KMP menjamin kompleksitas waktu linier sebesar $O(m)$ untuk tahap pembuatan tabel LPS dan $O(n)$ untuk tahap pencarian string, sehingga menghasilkan total kompleksitas waktu keseluruhan sebesar $O(n + m)$, dengan kebutuhan ruang bantu (*auxiliary space*) sebesar $O(m)$ untuk menyimpan tabel LPS.

2.4. Boyer-Moore

Algoritma Boyer-Moore merupakan metode pencarian string yang dikenal superior ketika diterapkan pada dokumen teks berskala besar. Keunikan utama dari algoritma ini terletak pada arah pemindaian karakternya yang dilakukan

secara terbalik, yaitu dari kanan ke kiri (*right-to-left*) dimulai dari ujung karakter pola paling akhir menuju karakter awal. Dengan menerapkan strategi pemindaian terbalik ini, Boyer-Moore dapat secara agresif memprediksi kecocokan dan melakukan lompatan besar (*skipping shifts*) melewati sebagian besar karakter teks yang tidak relevan. Konsep pergeseran ini dikontrol secara presisi melalui dua aturan heuristik utama, salah satunya adalah *Bad Character Rule* yang secara mandiri memproses karakter teks penyebab kegagalan pencocokan.

Dalam mempersiapkan proses pelompatan karakter, algoritma melakukan tahap pra-pemrosesan pola untuk membangun sebuah *Last Occurrence Table* (tabel kemunculan terakhir). Tabel ini bertindak sebagai pemetaan indeks yang mencatat posisi kemunculan terakhir dari setiap karakter unik di dalam pola pencarian. Apabila suatu karakter dalam alfabet tidak ditemukan sama sekali di dalam pola, tabel akan menyimpan nilai default berupa -1. Ketika perbandingan dari kanan ke kiri mendeteksi adanya ketidakcocokan karakter teks dengan pola yang disebut sebagai *bad character* program akan mencari posisi *bad character* tersebut di dalam *Last Occurrence Table*. Jika karakter tersebut ada di dalam pola, pola akan digeser ke kanan agar kemunculan terakhirnya selaras dengan karakter teks. Namun jika karakter tersebut sama sekali tidak terdaftar di dalam pola, pola akan digeser sepenuhnya melewati posisi ketidakcocokan saat itu.

Selama proses pemindaian berjalan, program melacak jumlah perbandingan karakter nyata melalui penambahan nilai pada variabel komparasi (comparison counting). Setiap operasi pengecekan karakter biner antara teks dan pola di dalam loop pergeseran akan dicatat untuk mengukur efisiensi kinerja secara riil di memori. Dari segi analisis efisiensi waktu, Boyer-Moore memiliki performa kasus terbaik (best-case) sebesar $O(n / m)$, di mana pola dapat melompat sejauh m karakter pada setiap kegagalan pencocokan. Sebaliknya, kasus terburuk (worst-case) yang dimilikinya bernilai $O(n \times m)$, yang umumnya terjadi apabila seluruh karakter di dalam teks dan pola memiliki nilai yang seragam.

2.5. Regular Expression (RegEx)

Regular Expression, atau disingkat RegEx, adalah urutan karakter khusus yang mendefinisikan suatu pola pencarian teks secara presisi dan efisien. Di dalam sistem komputasi modern, RegEx bertindak sebagai mesin pencari canggih yang mampu melakukan validasi format input, ekstraksi data semi-terstruktur, dan manipulasi string yang rumit. Kekuatan ekspresi reguler ini bertumpu pada kombinasi berbagai komponen dasar, seperti metakarakter, jangkar (anchors) untuk menentukan batasan posisi pencarian, kelas karakter (character classes) untuk mencocokkan kelompok simbol tertentu, serta kuantifier (quantifiers) untuk menentukan jumlah pengulangan karakter yang diharapkan.

Dalam rancangan aplikasi Judol Detector ini, RegEx dimanfaatkan secara khusus untuk mendeteksi pola spam kata kunci manipulatif leet-speak yang menggabungkan huruf dan angka secara langsung tanpa spasi (seperti “GACOR99” atau “MAXWIN88”). Pola pencocokan dinamis ini dibangun menggunakan ekspresi reguler kustom $[\backslash p\{L\}][\backslash p\{L\}\backslash p\{M\}]{2,}[0-9]{2,4}$ dengan beberapa penanda (flags) penting: flag g (global) untuk mencari seluruh kemunculan dalam halaman teks, flag i (case-insensitive) untuk mengabaikan perbedaan huruf kapital, serta flag u (unicode) yang dikombinasikan dengan token L (untuk huruf) dan M (untuk tanda baca/harakat). Konfigurasi ini menjamin bahwa sistem tetap sensitif dalam menangkap variasi karakter Unicode internasional sekaligus membatasi deteksi angka di bagian akhir sebanyak 2 hingga 4 digit.

Integrasi RegEx di dalam program dinilai sangat efisien karena memanfaatkan mesin eksekusi RegEx bawaan JavaScript (native engine) yang berjalan sangat cepat di level bahasa mesin dengan kompleksitas waktu rata-rata $O(N)$ di mana N adalah panjang teks. Guna meminimalisasi tingkat kesalahan deteksi (false positive), program juga dilengkapi dengan fungsi validasi tambahan `isSuspiciousToken()` yang bertugas menyaring token hasil pencocokan. Fungsi ini menganalisis struktur kata hasil pencarian dan memastikan bahwa token tersebut benar-benar mengandung sekurang-kurangnya 3 huruf alfabet dan 2 angka sebelum akhirnya diklasifikasikan sebagai konten mencurigakan yang harus ditandai dan dilaporkan pada popup statistik.

2.6. Weighted Levenshtein Distance

Dalam pencarian string pada dokumen, terdapat dua pendekatan utama yang sering digunakan, yaitu pencocokan presisi (*exact matching*) dan pencocokan perkiraan (*fuzzy matching*). Metode exact matching menuntut kesamaan karakter secara mutlak (kesamaan 100%), sehingga tidak sensitif terhadap variasi penulisan. Sebaliknya, metode fuzzy matching memberikan toleransi kesalahan penulisan dengan menghitung jarak transformasi kata berdasarkan tiga jenis operasi manipulasi dasar, yaitu penyisipan karakter baru (*insertion*), penghapusan karakter (*deletion*), dan penggantian karakter (*substitution*).

Weighted Levenshtein Distance sendiri adalah metrik untuk mengukur kemiripan string dengan mengukur jumlah minimum biaya operasi manipulasi tersebut yang diperlukan untuk mengubah satu string menjadi string lainnya. Dalam Weighted Levenshtein, bobot biaya untuk setiap operasi manipulasi dapat berbeda-beda berdasarkan probabilitas kesalahan, sebagai pengembangan dari versi standar yang menetapkan biaya konstan bernilai 1 untuk setiap operasi. Ini membuatnya sangat berguna dalam pemrosesan bahasa alami (NLP) dan OCR untuk menangani manipulasi visual karakter (seperti penggantian huruf o dengan angka

0, atau s dengan 5) dengan memberikan biaya substitusi yang lebih murah (misalnya 0.2 atau 0.3) sehingga sensitivitas deteksi manipulasi teks tetap tinggi.

Untuk menghitung jarak manipulasi secara efisien, digunakan Algoritma Wagner-Fischer yang menggunakan pendekatan pemrograman dinamis (*dynamic programming*). Algoritma ini membangun matriks jarak berukuran $(N+1) \times (M+1)$, di mana N dan M adalah panjang dari kedua string yang dibandingkan, dengan kompleksitas waktu sebesar $O(N \times M)$ dan kompleksitas ruang sebesar $O(N \times M)$. Setelah jarak manipulasi diperoleh, tingkat kemiripan (*similarity score*) dihitung secara ternormalisasi dengan rumus $1 - (\text{distance} / \max(\text{lengthA}, \text{lengthB}))$. Apabila kemiripannya memenuhi ambang batas tertentu (*similarity threshold*) sebesar 0.82, kata dinyatakan sebagai fuzzy match. Sebagai contoh, kata "H0KI88" terhadap kata kunci asli "HOKI88" sukses terdeteksi karena hanya memiliki satu substitusi berbiaya murah 0.2 dengan hasil kemiripan sebesar 96.7%.

Karakter A	Karakter B	Jenis Operasi	Bobot Biaya (Weight)	Rationale (Alasan Visual)
o	0		0.2	Bentuk visual mirip nol
i	1	Substitusi	0.2	Garis vertikal mirip angka satu
a	α	Substitusi	0.2	Alfa Yunani menyerupai 'a'
l	1	Substitusi	0.3	Huruf kecil 'L' mirip angka satu
a	4	Substitusi	0.3	Leet-speak pengganti 'a'
e	3	Substitusi	0.3	Leet-speak pengganti 'e'
s	5	Substitusi	0.3	Leet-speak pengganti 's'
Karakter Lain	Karakter Lain	Substitusi	1.0	Perbedaan karakter

				standar biasa
-	-	Insertion / Deletion	1.0	Penyisipan atau penghapusan standar

2.7. Optical Character Recognition (OCR)

Optical Character Recognition (OCR) adalah teknologi yang digunakan untuk mengenali karakter pada gambar dan mengubahnya menjadi teks yang dapat diproses oleh komputer. OCR memungkinkan teks yang tidak tersedia secara langsung pada struktur halaman web, seperti tulisan pada banner, poster, atau iklan berbentuk gambar, untuk dianalisis menggunakan algoritma pencocokan teks.

Secara umum, proses OCR terdiri atas beberapa tahap. Gambar masukan terlebih dahulu diproses untuk meningkatkan keterbacaan teks, misalnya melalui perubahan skala, penyesuaian kontras, konversi ke grayscale, atau binarisasi. Setelah itu, mesin OCR mengenali karakter pada gambar dan menghasilkan teks digital. Tahap preprocessing penting karena kualitas hasil pengenalan dipengaruhi oleh resolusi, tingkat kontras, orientasi teks, latar belakang, dan gangguan visual pada gambar.

Pada extension ini, OCR diimplementasikan menggunakan Tesseract.js, yaitu pustaka JavaScript yang membawa mesin Tesseract OCR ke lingkungan browser melalui WebAssembly. Proses pengenalan teks dijalankan secara asinkron menggunakan Web Worker agar komputasi OCR tidak secara langsung menghambat proses utama pada halaman web. Teks hasil ekstraksi kemudian diproses menggunakan algoritma pencocokan yang ada untuk menentukan apakah gambar mengandung konten judi online.

OCR berperan sebagai fitur pelengkap karena pemindaian DOM hanya dapat membaca teks yang tersimpan sebagai text node. Dengan adanya OCR, extension juga dapat mendeteksi teks yang sengaja disisipkan ke dalam gambar. Apabila ditemukan indikasi konten judi online, gambar terkait dapat diberi efek blur sehingga tidak terlihat secara langsung oleh pengguna. Meskipun demikian, hasil deteksi tetap bergantung pada kualitas dan keterbacaan gambar masukan.

2.8. Aho-Corasick

Algoritma Aho-Corasick adalah algoritma pencarian string berbasis otomatis (automaton) yang dirancang khusus untuk menyelesaikan masalah pencocokan

multi-pola (*multiple-pattern matching*) secara bersamaan. Jika algoritma konvensional seperti KMP harus memindai teks berulang kali untuk setiap kata kunci yang berbeda, Aho-Corasick mampu mencari seluruh kata kunci sekaligus hanya dalam satu kali pemindaian teks (*single pass*). Pondasi utama dari algoritma ini dibangun dengan menyusun seluruh kata kunci masukan ke dalam struktur data Trie (pohon kata pencarian). Di dalam Trie ini, setiap node merepresentasikan suatu state (keadaan) transisi karakter, dan setiap jalur dari akar (*root*) menuju node tertentu melambangkan awalan (*prefix*) dari satu atau beberapa kata kunci.

Setelah struktur Trie selesai dibangun, algoritma KMP memperluas pohon tersebut menjadi sebuah automaton utuh dengan menghitung fungsi kegagalan (Failure Link). Proses komputasi Failure Link ini dilakukan secara modular menggunakan algoritma penelusuran melebar (Breadth-First Search atau BFS) berbantuan Queue tingkat demi tingkat. Fungsi utama dari Failure Link adalah mengarahkan transisi pencarian ke state lain yang merupakan proper suffix (akhiran sejati) terpanjang dari substring yang sedang dicocokkan ketika terjadi kegagalan karakter (mismatch). Melalui mekanisme tautan kegagalan ini, pointer teks dapat terus bergerak maju tanpa pernah mengalami backtracking (mundur kembali), bahkan saat terjadi pergantian pencocokan kata kunci di tengah jalan.

Sebagai pelengkap transisi, algoritma ini membangun fungsi luaran (Output Link) pada setiap node untuk menyimpan daftar kata kunci yang sukses tercocok sepenuhnya pada state tersebut. Selama proses pencarian di halaman teks berjalan, algoritma Aho-Corasick mampu mempertahankan kompleksitas waktu yang sangat stabil sebesar $O(n + m + z)$, di mana n mewakili panjang teks, m mewakili total panjang seluruh kata kunci, dan z mewakili jumlah total kecocokan yang ditemukan. Dengan kebutuhan ruang memori (auxiliary space) sebesar $O(m \times \Sigma)$ (di mana Σ melambangkan ukuran alfabet), algoritma ini sangat optimal dan umum digunakan di dunia nyata untuk sistem deteksi intrusi jaringan (intrusion detection), ekstraksi teks (text mining), bioinformatika, dan deteksi plagiarisme.

2.9. Rabin-Karp

Algoritma Rabin-Karp adalah algoritma pencarian string yang memanfaatkan fungsi matematis berupa nilai hash untuk mempercepat pencarian pola di dalam teks. Berbeda dari metode naif (Naive Algorithm) yang membandingkan setiap karakter string satu per satu secara langsung sejak awal, Rabin-Karp mempercepat eliminasi ketidakcocokan dengan membandingkan nilai numerik hash pola terhadap nilai hash dari setiap jendela substring teks. Efisiensi luar biasa ini dicapai melalui penggunaan fungsi Rolling Hash, yang memungkinkan nilai hash dari jendela teks berikutnya dihitung hanya dalam waktu konstan $O(1)$ berdasarkan nilai hash jendela sebelumnya. Rumus perhitungan rolling hash ini

dilakukan dengan membuang kontribusi nilai karakter paling kiri yang keluar jendela dan menambahkan kontribusi karakter baru yang masuk jendela menggunakan rumus matematis:

$$H[i] = xH[i+1] + (T[i] - T[i+|P|]x^{|P|}) \bmod p$$

Di dalam persamaan tersebut, $H[i]$ merupakan nilai hash dari jendela teks pada indeks i , sedangkan $H[i+1]$ adalah nilai hash pada jendela teks adjacent (di sebelah kanan). Variabel x merepresentasikan ukuran basis alfabet (dalam program bernilai 256 untuk ASCII), dan $T[i]$ adalah karakter teks pada indeks i yang dimasukkan ke dalam hash. Variabel $T[i + |P|]$ merepresentasikan karakter teks yang dikeluarkan dari jendela pencarian (di mana $|P|$ melambangkan panjang pola pencarian), sedangkan $x^{|P|}$ adalah nilai pengali eksponensial tinggi, dan p merupakan bilangan prima besar yang digunakan sebagai modulus (dalam program bernilai 101).

Penggunaan operasi modulo dengan bilangan prima sangat krusial untuk mencegah terjadinya luapan integer (integer overflow) sekaligus menekan seminimal mungkin probabilitas kesamaan nilai hash untuk string yang berbeda. Namun, karena dua string berbeda secara teoritis tetap dapat menghasilkan nilai hash yang identik akibat keterbatasan rentang modulo sebuah fenomena yang disebut kolisi hash (hash collision) maka kesamaan nilai hash tidak langsung menjamin kecocokan string yang sebenarnya. Oleh karena itu, ketika program mendeteksi kesamaan nilai hash pola dan jendela teks, program wajib menjalankan verifikasi kecocokan karakter secara manual dari kiri ke kanan demi menjamin akurasi mutlak.

Dari segi efisiensi, Rabin-Karp memiliki kompleksitas waktu kasus rata-rata (average case) sebesar $O(n + m)$, karena proses peluncuran rolling hash berjalan sangat cepat dan tahap verifikasi manual hanya dipicu pada momen yang sangat jarang. Namun pada kasus terburuk (worst case) di mana kolisi hash terjadi pada setiap jendela teks, kompleksitasnya dapat menurun menjadi $O(n \times m)$. Dengan kebutuhan ruang bantu (auxiliary space) yang sangat hemat sebesar $O(1)$, Rabin-Karp sangat ideal digunakan dalam mendeteksi plagiarisme dokumen, perbandingan substring duplikat, dan pencarian multi-pola berskala besar.

3. Analisis Pemecahan Masalah

3.1. Alur Kerja Sistem

SweetBonanza Judol Detector bekerja sebagai Chromium browser extension yang memindai konten halaman web secara otomatis. Ketika suatu halaman dibuka,

content script dijalankan dan memuat daftar kata kunci dari berkas keywords.txt. Setiap baris pada berkas tersebut diperlakukan sebagai sebuah pola pencarian. Sebelum pemindaian dimulai, extension menghapus hasil pemindaian sebelumnya dan mengatur ulang statistik agar data yang ditampilkan selalu sesuai dengan kondisi halaman terbaru.

Pemindaian teks dilakukan dengan menelusuri text node pada struktur Document Object Model (DOM). Extension menggunakan TreeWalker untuk mengambil node yang memiliki konten teks serta mengabaikan elemen yang tidak relevan, seperti script, style, input, textarea, code, dan pre. Pendekatan ini memungkinkan sistem menganalisis teks yang terlihat pada halaman tanpa mengganggu elemen yang memiliki fungsi khusus.

Pada setiap text node, sistem menjalankan beberapa metode pencocokan. Algoritma Knuth-Morris-Pratt dan Boyer-Moore digunakan untuk mencari kemunculan kata kunci secara tepat (exact matching). Regular Expression digunakan untuk mendeteksi pola gabungan kata dan angka, seperti GACOR99 atau MAXWIN234. Weighted Levenshtein Distance digunakan untuk menemukan variasi kata yang memiliki tingkat kemiripan tinggi dengan kata kunci, termasuk teks yang disamarkan melalui penggantian karakter yang serupa secara visual. Hasil dari seluruh algoritma kemudian digabungkan. Apabila beberapa algoritma mendeteksi bagian teks yang sama, hasil tersebut dikelompokkan agar elemen hanya ditandai satu kali.

Teks yang terdeteksi dibungkus menggunakan elemen <mark> dan diberi warna penanda agar mudah dikenali. Setiap elemen hasil deteksi menyimpan informasi mengenai keyword dan algoritma yang menemukan kecocokan. Ketika pengguna mengarahkan kursor ke elemen tersebut, extension menampilkan tooltip yang berisi keyword, algoritma, jumlah kemunculan, dan waktu eksekusi. Apabila fitur blur diaktifkan, teks yang telah ditandai juga diberi efek buram.

Setelah pemindaian selesai, statistik disimpan menggunakan chrome.storage.local. Statistik tersebut mencakup total hasil deteksi, frekuensi setiap keyword, jumlah kecocokan tiap algoritma, jumlah perbandingan karakter, waktu eksekusi, serta informasi halaman yang dipindai. Popup extension membaca data tersebut untuk menampilkan ringkasan pemindaian secara real-time. Pengguna juga dapat menjalankan pemindaian ulang melalui tombol rescans pada popup.

Extension juga menangani perubahan konten pada halaman dinamis menggunakan MutationObserver. Apabila terjadi perubahan pada DOM yang

berasal dari halaman web, sistem menjadwalkan pemindaian ulang setelah jeda singkat untuk menghindari pemrosesan berulang secara berlebihan. Selain itu, apabila fitur OCR diaktifkan, extension memproses gambar yang memenuhi kriteria tertentu menggunakan Tesseract.js. Teks hasil OCR kemudian diperiksa dengan metode pencocokan yang sama. Gambar yang terdeteksi mengandung indikasi judi online diberi efek blur sebagai bentuk penyensoran visual.

3.2. Preprocessing dan Normalisasi Teks

Sebelum proses pencocokan dilakukan, extension terlebih dahulu memuat daftar kata kunci dari file keywords.txt. Isi file dipisahkan berdasarkan pergantian baris, kemudian setiap kata kunci dibersihkan dari spasi pada bagian awal dan akhir menggunakan operasi trim. Baris kosong dihapus agar tidak diproses sebagai pola pencarian. Setiap baris yang tersisa diperlakukan sebagai satu keyword, termasuk frasa yang terdiri atas beberapa kata seperti slot gacor atau bonus new member.

Konten halaman web diambil melalui penelusuran text node pada struktur DOM menggunakan TreeWalker. Sistem hanya memproses node yang memiliki teks dan mengabaikan elemen yang tidak relevan, yaitu script, style, noscript, mark, input, textarea, code, dan pre. Elemen yang dapat diedit oleh pengguna melalui atribut contentEditable juga tidak dipindai. Penyaringan ini mencegah pemrosesan kode program, masukan pengguna, serta elemen highlight yang sebelumnya dibuat oleh extension.

Pada pencarian menggunakan KMP dan Boyer-Moore, teks dan keyword diubah menjadi huruf kecil sebelum dibandingkan. Normalisasi ini membuat pencarian bersifat case-insensitive, sehingga variasi kapitalisasi seperti SLOT, Slot, dan slot diperlakukan sebagai teks yang sama. Meskipun demikian, sistem tetap mempertahankan indeks pada teks asli agar highlight dapat diterapkan secara tepat tanpa mengubah bentuk teks yang ditampilkan pada halaman.

Pada fuzzy matching, teks dipecah menjadi token menggunakan pola yang mendukung huruf Unicode, combining mark, dan angka. Token yang diproses memiliki panjang minimal tiga karakter. Sebelum dibandingkan menggunakan Weighted Levenshtein Distance, sistem menyaring pasangan token dan keyword berdasarkan selisih panjang agar komputasi tetap efisien. Perbandingan hanya dilakukan apabila selisih panjang keduanya tidak melebihi dua karakter atau 40% dari panjang keyword. Token dan keyword kemudian diubah menjadi huruf kecil sebelum nilai kemiripannya dihitung.

Sistem tidak menghapus tanda baca atau mengubah struktur teks secara menyeluruh karena posisi karakter asli dibutuhkan untuk menentukan lokasi highlight pada DOM. Dengan demikian, proses preprocessing tetap menjaga keseimbangan antara konsistensi pencocokan, efisiensi komputasi, dan ketepatan penandaan elemen pada halaman web. Teks hasil ekstraksi OCR juga diproses menggunakan tahapan pencocokan yang sama sebelum gambar diklasifikasikan sebagai konten yang perlu disensor.

3.3. Penerapan KMP dan Boyer-Moore

Algoritma Knuth-Morris-Pratt (KMP) dan Boyer-Moore digunakan untuk melakukan *exact matching* pada konten teks di halaman web. Pada setiap *text node* yang telah melalui proses penyaringan, isi node diperlakukan sebagai teks pencarian, sedangkan setiap baris pada berkas keywords.txt diperlakukan sebagai pola. Pencarian dilakukan secara iteratif ke seluruh keyword. Sebelum dibandingkan, teks dan keyword diubah menjadi huruf kecil agar pencarian tidak dipengaruhi oleh perbedaan kapitalisasi. Teks asli tetap dipertahankan agar posisi hasil pencocokan dapat digunakan untuk memberikan highlight secara tepat.

Pada penerapan KMP, sistem terlebih dahulu membangun tabel “Longest Proper Prefix which is also Suffix” atau LPS untuk setiap keyword. Ketika terjadi ketidakcocokan karakter, tabel tersebut digunakan untuk menentukan posisi pola berikutnya tanpa mengulang pemindaian karakter teks yang telah diperiksa. Setiap kecocokan disimpan dalam bentuk indeks awal dan indeks akhir pada teks. Setelah satu kecocokan ditemukan, pencarian tetap dilanjutkan dengan memanfaatkan nilai LPS agar seluruh kemunculan keyword, termasuk kemunculan yang saling tumpang tindih, dapat terdeteksi.

Pada penerapan Boyer-Moore, sistem membangun *last occurrence table* untuk mencatat posisi terakhir setiap karakter pada keyword. Proses pencocokan dilakukan dari karakter paling kanan menuju kiri. Apabila terjadi ketidakcocokan, pola digeser berdasarkan posisi karakter yang menyebabkan kegagalan pencocokan. Strategi ini memungkinkan sistem melewati bagian teks yang tidak relevan tanpa memeriksa setiap posisi secara berurutan. Setelah satu kecocokan ditemukan, pola digeser satu posisi agar kemunculan lain yang berdekatan tetap dapat dideteksi.

KMP dan Boyer-Moore dijalankan secara berurutan pada setiap *text node*. Masing-masing algoritma menghasilkan daftar kecocokan yang memuat keyword, potongan teks asli, indeks awal, indeks akhir, serta nama algoritma. Sistem juga mencatat jumlah perbandingan karakter dan waktu eksekusi untuk keperluan statistik dan analisis performa.

Hasil dari kedua algoritma kemudian digabungkan dengan hasil RegEx dan Weighted Levenshtein Distance. Apabila beberapa algoritma mendeteksi rentang teks yang sama, sistem mengelompokkan hasil tersebut agar teks hanya diberi satu highlight. Informasi seluruh algoritma yang mendeteksi teks tersebut tetap disimpan dan ditampilkan pada tooltip. Dengan ini, KMP dan Boyer-Moore tidak hanya digunakan sebagai metode deteksi exact match, tetapi juga menjadi dasar perbandingan efisiensi algoritma pada karakteristik teks yang berbeda.

3.4. Penerapan RegEx

Regular Expression (RegEx) digunakan sebagai metode tambahan untuk mendeteksi pola teks yang tidak bergantung pada daftar keyword dalam berkas keywords.txt. Metode ini ditujukan untuk menangkap token yang terdiri atas rangkaian huruf dan diikuti oleh angka tanpa spasi, seperti GACOR99, MAXWIN234, dan SLOT88. Pola semacam ini sering digunakan untuk membentuk nama promosi atau domain yang menyerupai konten judi online.

Pola RegEx yang digunakan pada program adalah sebagai berikut.

```
/[\p{L}][\p{L}\p{M}]{2,}[0-9]{2,4}/giu
```

Pola tersebut memiliki syarat yaitu sedikitnya tiga huruf yang langsung diikuti oleh dua hingga empat angka. Notasi `\p{L}` digunakan untuk mengenali huruf Unicode, sedangkan `\p{M}` memungkinkan sistem menangani combining mark yang dapat muncul pada karakter tertentu. Flag `g` digunakan untuk mencari seluruh kemunculan dalam teks, flag `i` membuat pencarian tidak membedakan huruf kapital dan huruf kecil, sedangkan flag `u` mengaktifkan dukungan Unicode.

RegEx dijalankan terhadap setiap text node yang telah melalui proses penyaringan DOM. Sebelum pencarian dimulai, posisi pencarian RegEx diatur ulang agar setiap node diproses dari indeks awal. Seluruh kecocokan kemudian diambil menggunakan `matchAll`. Untuk setiap hasil, sistem menyimpan teks yang ditemukan, indeks awal, indeks akhir, serta label algoritma RegEx.

Program juga menjalankan validasi tambahan melalui fungsi `isSuspiciousToken()`. Fungsi ini memastikan bahwa token hasil pencocokan memiliki setidaknya tiga huruf dan dua angka. Validasi tersebut menjaga konsistensi hasil sebelum token dimasukkan ke dalam daftar kecocokan dan ditampilkan pada halaman.

Selain mencatat jumlah kecocokan, sistem mengukur waktu eksekusi RegEx menggunakan `performance.now()`. Hasil RegEx kemudian digabungkan dengan

hasil KMP, Boyer-Moore, dan Weighted Levenshtein Distance. Apabila beberapa algoritma menemukan rentang teks yang sama, sistem mengelompokkannya agar hanya satu highlight ditampilkan pada halaman.

Pola RegEx yang digunakan memiliki batasan berupa urutan huruf dan angka yang harus bersebelahan secara langsung. Token dengan pemisah seperti spasi, tanda hubung, atau garis bawah tidak terdeteksi melalui RegEx ini. Namun, variasi tersebut masih dapat dideteksi melalui pencarian keyword atau fuzzy matching apabila memiliki kemiripan yang mencukupi.

3.5. Penerapan Weighted Levenshtein Distance

Weighted Levenshtein Distance digunakan untuk mendeteksi variasi kata kunci yang tidak dapat ditemukan melalui pencocokan eksak. Metode ini diperlukan karena konten judi online sering menggunakan penyamaran teks, seperti mengganti huruf o dengan angka 0, huruf i dengan angka 1, atau huruf a dengan karakter Unicode α . Sebagai contoh, kata H0KI88 tetap dapat dikenali sebagai variasi dari HOKI88 meskipun keduanya tidak memiliki susunan karakter yang sepenuhnya sama.

Sebelum proses pencocokan dilakukan, teks pada setiap text node dipecah menjadi sejumlah token kandidat. Setiap token kemudian dibandingkan dengan seluruh keyword pada berkas keywords.txt. Untuk mengurangi komputasi yang tidak diperlukan, sistem hanya membandingkan token dan keyword yang memiliki selisih panjang dalam batas tertentu. Pada setiap pasangan yang memenuhi syarat, program menghitung nilai kemiripan berdasarkan Weighted Levenshtein Distance.

Berbeda dengan Levenshtein Distance standar, biaya substitusi karakter pada metode ini tidak selalu bernilai satu. Pasangan karakter yang memiliki kemiripan visual diberi biaya lebih kecil, misalnya o dan 0 sebesar 0.2, i dan 1 sebesar 0.2, serta a dan α sebesar 0.2. Sementara itu, penggantian karakter biasa serta operasi penyisipan dan penghapusan tetap menggunakan biaya standar. Nilai jarak yang diperoleh kemudian dinormalisasi menjadi similarity score dengan rumus berikut.

$$\text{similarity} = 1 - (\text{distance} / \max(\text{length}(\text{token}), \text{length}(\text{keyword})))$$

Untuk setiap token, sistem memilih keyword dengan nilai kemiripan tertinggi. Token diklasifikasikan sebagai fuzzy match apabila nilainya mencapai ambang batas 0.82 dan token tersebut tidak identik dengan keyword setelah dinormalisasi

menjadi huruf kecil. Pengecualian terhadap kecocokan identik dilakukan agar hasil exact matching tidak dilaporkan kembali sebagai fuzzy matching.

Setiap hasil pencocokan menyimpan keyword acuan, teks aktual yang ditemukan, indeks awal, indeks akhir, nama algoritma Weighted-Levenshtein, serta nilai kemiripannya. Waktu eksekusi juga dicatat untuk ditampilkan pada statistik popup. Hasil tersebut kemudian digabungkan dengan hasil KMP, Boyer-Moore, dan RegEx sebelum elemen yang terdeteksi diberi highlight pada halaman web.

3.6. Pengolahan DOM

Pengolahan Document Object Model (DOM) dilakukan agar extension dapat membaca dan menandai teks pada halaman web tanpa merusak struktur maupun tata letaknya. Proses diawali dengan penelusuran text node menggunakan TreeWalker. Sistem hanya mengambil node yang memiliki konten teks dan mengabaikan elemen yang tidak relevan, seperti script, style, noscript, input, textarea, code, pre, serta elemen yang dapat diedit oleh pengguna. Elemen highlight yang dibuat oleh extension juga dikecualikan agar tidak diproses kembali pada pemindaian berikutnya.

Setiap text node yang lolos penyaringan diproses menggunakan algoritma pencocokan yang tersedia. Hasil pencocokan dari beberapa algoritma dapat memiliki rentang indeks yang sama atau saling tumpang tindih. Oleh karena itu, sistem terlebih dahulu mengelompokkan dan menyaring hasil deteksi sebelum melakukan perubahan pada DOM. Langkah ini mencegah munculnya highlight berlapis pada bagian teks yang sama.

Teks yang terdeteksi dibungkus menggunakan elemen <mark>. Pemisahan teks dilakukan berdasarkan indeks awal dan indeks akhir hasil pencocokan, kemudian bagian-bagian tersebut disusun kembali menggunakan DocumentFragment. Pendekatan ini memungkinkan extension memberikan highlight hanya pada potongan teks yang relevan tanpa mengganti seluruh isi elemen melalui innerHTML. Dengan demikian, struktur halaman, event listener, dan tata letak asli tetap terjaga.

Setiap elemen <mark> menyimpan informasi tambahan melalui atribut data-*, seperti keyword, algoritma yang mendeteksi kecocokan, dan nilai kemiripan apabila hasil berasal dari Weighted Levenshtein Distance. Informasi tersebut digunakan untuk menampilkan tooltip ketika pengguna mengarahkan kursor ke teks yang ditandai. Apabila fitur blur diaktifkan, elemen highlight juga diberi kelas CSS tambahan untuk menghasilkan efek buram.

Sebelum pemindaian ulang dilakukan, extension menghapus seluruh elemen `<mark>` yang dibuat pada pemindaian sebelumnya. Isi teks pada setiap elemen dikembalikan ke parent node, kemudian dilakukan normalisasi node untuk menggabungkan kembali potongan teks yang terpisah. Tahap pembersihan ini memastikan bahwa hasil rescans selalu merepresentasikan kondisi halaman terbaru dan mencegah penumpukan elemen tambahan pada DOM.

Untuk menangani halaman web dinamis, extension menggunakan `MutationObserver` untuk mendeteksi perubahan struktur halaman. Ketika terdapat perubahan yang relevan, sistem menjadwalkan pemindaian ulang setelah jeda singkat. Mekanisme ini membuat extension tetap responsif terhadap konten baru tanpa menjalankan proses pencocokan secara berulang dalam waktu yang terlalu rapat.

3.7. Statistik pada Popup Extension

Popup extension digunakan sebagai antarmuka utama untuk menampilkan ringkasan hasil pemindaian halaman web. Setelah proses deteksi selesai, content script mengumpulkan statistik dari seluruh algoritma dan menyimpannya melalui `chrome.storage.local`. Data tersebut kemudian dibaca oleh popup sehingga pengguna dapat melihat hasil pemindaian tanpa perlu membuka developer tools.

Informasi yang ditampilkan mencakup jumlah total teks yang terdeteksi, frekuensi kemunculan setiap keyword, jumlah kecocokan yang ditemukan oleh masing-masing algoritma, jumlah perbandingan karakter, serta waktu eksekusi. Popup juga menampilkan informasi halaman yang sedang dipindai, seperti judul halaman dan URL. Statistik tersebut membantu pengguna memahami hasil deteksi sekaligus membandingkan performa setiap metode pencocokan.

Data pada popup diperbarui secara real-time ketika nilai pada `chrome.storage.local` berubah. Dengan mekanisme ini, hasil pemindaian terbaru dapat langsung ditampilkan setelah halaman dibuka, ketika konten halaman berubah, atau setelah pengguna menjalankan pemindaian ulang. Pendekatan tersebut juga memisahkan proses pemindaian dari antarmuka popup sehingga keduanya dapat berjalan secara independen.

Selain menampilkan statistik, popup menyediakan tombol rescans untuk memicu pemindaian ulang pada halaman aktif. Popup juga menyediakan pengaturan untuk mengaktifkan atau menonaktifkan fitur blur teks dan OCR gambar. Setiap perubahan pengaturan disimpan pada `chrome.storage.local` dan diterapkan kembali oleh content script pada halaman web.

Melalui popup extension, pengguna memperoleh akses terpusat terhadap hasil deteksi, informasi performa algoritma, dan konfigurasi fitur tambahan dalam satu antarmuka yang ringkas.

3.8. Fitur Bonus

3.8.1. Blur Teks

Fitur blur teks digunakan untuk menyensor konten yang terdeteksi mengandung teks yang berhubungan dengan judi online. Berbeda dari highlight yang hanya memberikan penanda visual, fitur ini memburamkan teks agar tidak dapat dibaca secara langsung. Pengguna tetap dapat mengetahui lokasi konten mencurigakan tanpa terpapar isi teks secara jelas.

Fitur ini dapat diaktifkan atau dinonaktifkan melalui toggle Blur teks pada popup extension. Status pengaturan disimpan menggunakan `chrome.storage.local` sehingga pilihan pengguna tetap tersimpan ketika popup ditutup atau halaman dimuat ulang. Perubahan status juga dipantau oleh content script agar efek blur dapat diterapkan secara langsung tanpa memerlukan pemindaian ulang.

Ketika fitur diaktifkan, setiap elemen `<mark>` hasil deteksi diberi efek CSS filter: `blur(4px)`. Efek tersebut diterapkan baik pada highlight yang telah tersedia maupun pada highlight baru yang dibuat setelah proses rescanning. Apabila fitur dinonaktifkan, properti blur dihapus sehingga teks kembali terlihat, sedangkan highlight tetap dipertahankan.

Fitur blur teks bersifat terpisah dari OCR gambar. Blur teks hanya diterapkan pada teks hasil pemindaian DOM, sedangkan gambar yang terdeteksi melalui OCR ditangani menggunakan mekanisme penyensoran visual tersendiri.

3.8.2. OCR Gambar

Fitur OCR gambar digunakan untuk mendeteksi konten judi online yang disisipkan dalam bentuk gambar, seperti banner, poster, atau iklan. Teks pada gambar tidak tersimpan sebagai text node pada DOM sehingga tidak dapat dibaca melalui pemindaian teks biasa. Oleh karena itu, extension menggunakan Tesseract.js untuk mengekstrak teks dari gambar sebelum menjalankan algoritma pencocokan.

Fitur ini dapat diaktifkan atau dinonaktifkan melalui toggle OCR pada popup extension. Ketika fitur diaktifkan, sistem memilih elemen gambar yang terlihat pada halaman dan memiliki ukuran minimum tertentu. Penyaringan ini dilakukan agar gambar berukuran kecil, tersembunyi, atau tidak relevan tidak diproses secara berlebihan. Untuk menjaga performa, pemrosesan dilakukan secara bertahap dengan jumlah maksimum 12 gambar pada setiap batch.

Sebelum diproses oleh Tesseract.js, gambar dikonversi menjadi data URL dan melalui tahap preprocessing. Sistem menyesuaikan ukuran gambar, mengubah warna menjadi grayscale, serta meningkatkan kontras untuk memperjelas teks. Apabila gambar berasal dari sumber eksternal, content script meminta bantuan background service worker untuk mengambil data gambar. Sistem juga dapat memindai screenshot dari area tab yang sedang terlihat untuk mendeteksi tulisan pada elemen visual yang tidak dapat diakses secara langsung.

Teks hasil ekstraksi OCR diperiksa menggunakan algoritma KMP, Boyer-Moore, RegEx, dan Weighted Levenshtein Distance. KMP dan Boyer-Moore digunakan untuk mencari keyword secara tepat, RegEx digunakan untuk mendeteksi pola gabungan kata dan angka, sedangkan Weighted Levenshtein Distance menangani variasi karakter yang disamarkan. Hasil pencocokan kemudian disaring untuk menghapus duplikasi sebelum dicatat sebagai hasil deteksi OCR.

Apabila ditemukan indikasi konten judi online, gambar terkait diberi efek blur(8px) serta garis tepi berwarna merah agar lokasinya tetap dapat dikenali. Informasi keyword yang terdeteksi juga disimpan pada atribut elemen gambar dan ditampilkan melalui tooltip bawaan browser ketika kursor diarahkan ke gambar tersebut. Jumlah kecocokan dan waktu eksekusi OCR turut dicatat dalam statistik popup.

Pemindaian OCR dapat dijalankan kembali ketika halaman dimuat, ketika pengguna melakukan scroll atau mengubah ukuran jendela, serta ketika terdapat gambar baru atau perubahan sumber gambar pada DOM. Mekanisme ini memungkinkan extension menangani halaman dinamis tanpa melakukan pemrosesan gambar secara terus-menerus.

3.8.3. Aho-Corasick dan Rabin-Karp

Dua algoritma bonus, yaitu Aho-Corasick dan Rabin-Karp, telah berhasil diimplementasikan dan diintegrasikan ke dalam sistem pemindaian halaman web pada content script. Penambahan kedua algoritma ini tidak hanya untuk kriteria bonus, tetapi juga memberikan alternatif pencarian multi-pola berkinerja tinggi serta metode verifikasi berbasis numerik hash yang menambahkan variasi pengujian performa sistem.

Pada implementasi Aho-Corasick, proses deteksi dibagi menjadi dua tahap utama, yaitu pembangunan mesin pencocokan (automaton) dan pemindaian teks. Tahap inisialisasi diawali dengan pembentukan struktur Trie di mana setiap kata kunci dari keywords.txt dimasukkan sebagai state transisi karakter. Setelah Trie terbentuk, fungsi kegagalan (*Failure Link*) dihitung secara asinkron menggunakan BFS dengan bantuan Queue tingkat demi tingkat. Failure Link ini bekerja dengan mengarahkan pointer ke state transisi terdalam yang merepresentasikan akhiran sejati (proper suffix) terpanjang dari pola jika terjadi kegagalan karakter (mismatch). Selama penelusuran teks halaman berjalan, mesin automaton Aho-Corasick dapat memindai seluruh kata kunci judi online secara sekaligus hanya dalam satu kali jalan (single pass) dengan kompleksitas waktu sangat efisien sebesar $O(n + m + z)$, di mana n adalah panjang teks, m adalah total panjang kata kunci, dan z adalah jumlah kecocokan yang ditemukan.

Sementara itu, pada implementasi Rabin-Karp, pencarian pola dilakukan dengan memanfaatkan representasi numerik hash dari string untuk mempercepat proses eliminasi ketidakcocokan. Program menetapkan basis alfabet $d = 256$ (untuk mendukung karakter ASCII penuh) dan konstanta prima $q = 101$ sebagai nilai modulus guna meminimalisasi terjadinya kolisi hash (hash collision). Sebelum jendela pencarian digeser di atas teks, program terlebih dahulu melakukan prakomputasi terhadap pengali tingkat tinggi. Nilai hash awal untuk pola dan jendela pertama teks dihitung secara linier, kemudian jendela digeser dari kiri ke kanan di sepanjang teks halaman menggunakan fungsi Rolling Hash. Ketika jendela bergeser dari indeks i ke indeks $i+1$, nilai hash diperbarui dalam kompleksitas waktu konstan $O(1)$ melalui persamaan matematis $t = (d * (t - \text{text.charCodeAt}(i) * h) + \text{text.charCodeAt}(i + m)) \% q$. Program juga dilengkapi dengan penanganan khusus nilai hash negatif menggunakan instruksi `if (t < 0) t += q`, untuk memastikan hasil operasi modulo tetap bernilai positif. Karena terdapat kemungkinan kolisi di mana dua string

yang berbeda menghasilkan nilai hash yang identik, program wajib menjalankan verifikasi karakter secara manual dari kiri ke kanan (character-by-character verification) apabila nilai hash pola dan jendela teks terdeteksi sama persis. Hal ini menjamin tingkat akurasi 100% bebas dari kesalahan positif palsu (false positive).

4. Implementasi dan Pengujian

4.1. Lingkungan Pengembangan

Judol Detector ini dikembangkan sebagai Chromium browser extension menggunakan TypeScript. Kompilasi dan bundling dilakukan dengan Vite, sedangkan pengelolaan dependensi menggunakan npm. Extension menggunakan Manifest V3 dan terdiri atas tiga komponen utama, yaitu content script untuk memindai serta memodifikasi halaman web, popup untuk menampilkan statistik dan pengaturan, serta background service worker untuk mendukung pemrosesan gambar.

Fitur Optical Character Recognition (OCR) diimplementasikan menggunakan Tesseract.js. Pada saat proses build, aset pendukung Tesseract.js disalin ke direktori dist/ agar dapat diakses secara lokal oleh extension. Hasil kompilasi kemudian dimuat ke browser melalui fitur Load unpacked.

Perangkat lunak utama yang digunakan dirangkum pada Tabel berikut.

Komponen	Versi	Keterangan
Typescript	^6.0.3	Bahasa pemrograman utama untuk pengembangan extension.
Node.js	Menyesuaikan lingkungan pengembangan	Runtime untuk menjalankan proses <i>build</i> .
npm	Menyesuaikan lingkungan pengembangan	Pengelola dependensi proyek.
Vite	^8.0.14	Build tool untuk menghasilkan berkas

		extension pada direktori dist/.
Chromium-based browser	Menyesuaikan browser pengujian	Platform untuk menjalankan extension, seperti Google Chrome, Chromium, Brave, atau Microsoft Edge.
Manifest V3	3	Format konfigurasi Chromium browser extension.
Tesseract.js	^7.0.0	Pustaka OCR untuk mengekstrak teks dari gambar.
Git	Menyesuaikan lingkungan pengembangan	Version control system untuk program ini.

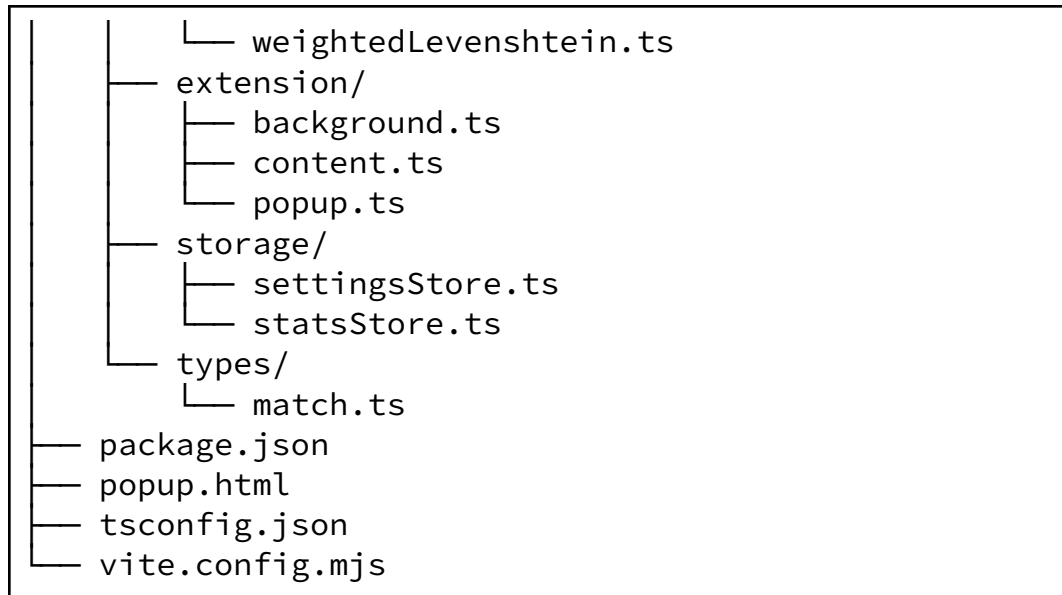
Proyek dirancang agar dapat dibangun pada sistem operasi Windows dan Linux. Proses kompilasi dilakukan dengan perintah berikut.

```
npm install
npm run build
```

Hasil kompilasi disimpan pada direktori dist/ dan dapat dimuat secara langsung sebagai extension pada browser berbasis Chromium.

4.2. Struktur Program

```
Tubes3_SweetBonanza/
├── dist/
├── keywords/
│   └── keywords.txt
├── public/
│   └── manifest.json
├── scripts/
│   └── build.mjs
├── src/
│   ├── algorithms/
│   │   ├── kmp.ts
│   │   ├── boyerMoore.ts
│   │   └── RegexMatcher.ts
```



Fungsi pada setiap bagian utama dirangkum pada tabel berikut:

Direktori atau File	Keterangan
keywords/keywords.txt	Menyimpan daftar kata kunci judi online. Setiap kata kunci dipisahkan oleh baris baru dan digunakan sebagai pola pencarian oleh algoritma string matching.
public/manifest.json	Mendefinisikan konfigurasi extension berdasarkan Manifest V3, termasuk permission, content script, popup, background service worker, dan aset OCR yang dapat diakses extension.
src/algorithms/	Menyimpan implementasi algoritma pencocokan teks, yaitu KMP, Boyer-Moore, RegEx, dan Weighted Levenshtein Distance.
src/extension/content.ts	Menjadi komponen utama yang berjalan pada halaman web. Modul ini memindai DOM, menjalankan algoritma pencocokan, memberikan highlight, menampilkan tooltip, menerapkan blur, dan menjalankan OCR.
src/extension/background.ts	Menjadi background service worker yang mendukung pengambilan data gambar dan screenshot halaman untuk pemrosesan OCR.
src/extension/	Mengatur antarmuka popup, menampilkan

popup.ts	statistik hasil pemindaian, mengirim perintah rescan, serta mengelola toggle OCR dan blur teks.
src/storage/settingsStore.ts	Menyimpan dan membaca pengaturan OCR serta blur teks menggunakan chrome.storage.local.
src/storage/statsStore.ts	Menyimpan statistik hasil pemindaian dan memperbarui data popup secara real-time.
src/types/match.ts	Mendefinisikan tipe data bersama, seperti hasil pencocokan dan statistik setiap algoritma.
scripts/build.mjs	Menjalankan proses build dan menyalin aset Tesseract.js ke dalam direktori dist/.
popup.html	Mendefinisikan struktur dan tampilan dasar antarmuka popup extension.
dist/	Menyimpan hasil build yang siap dimuat ke browser menggunakan fitur <i>Load unpacked</i> .

4.3. Implementasi Algoritma

4.3.1. Knuth-Morris-Pratt'

Fungsi utama eksekusi KMP adalah searchKMP(text, keywords). Fungsi ini menerima teks dan array keywords, lalu mengiterasi setiap kata kunci untuk dicocokkan ke dalam teks menggunakan kmpSearch. Seluruh teks dan kata kunci dinormalisasi ke huruf kecil sebelum pencarian dilakukan, sehingga proses bersifat case-insensitive.

Sebelum pencarian dimulai, fungsi tableLPS(pattern) membangun array LPS. Array ini berukuran sama dengan panjang pattern, dan setiap indeks i menyimpan panjang prefix terpanjang dari substring pattern[0..i] yang sekaligus merupakan suffix-nya. Ini bertujuan untuk menghindari perbandingan ulang karakter yang sudah diketahui cocok, sehingga kompleksitas tetap $O(m)$ untuk pembangunan tabel.

Pada kmpSearch(text, pattern), dua pointer digunakan: i untuk teks dan j untuk pattern.

Adapun penghitungan perbandingan dilakukan di dua tempat:

- Saat membangun LPS. Setiap iterasi while ($i < \text{pattern.length}$) menambah comparisons.

- Saat pencarian. Setiap iterasi while ($i < \text{text.length}$) menambah comparisons.

Total comparisons dari kedua fase digabungkan dan dilaporkan di stats, memberikan gambaran nyata biaya komputasi per kata kunci.

Berikut adalah implementasi dalam bahasa *Typescript*.

```
function kmpSearch(text: string, pattern: string): { indexes:
number[]; comparisons: number } {
    if (pattern.length === 0) return { indexes: [], comparisons: 0 };
    const { lps, comparisons: lpsComparisons } = tableLPS(pattern);
    const result: number[] = [];
    let comparisons = lpsComparisons;
    let i = 0;
    let j = 0;
    while (i < text.length) {
        comparisons++;
        if (text[i] === pattern[j]) {
            i++;
            j++;
        }
        if (j === pattern.length) {
            result.push(i - j);
            j = lps[j - 1];
        }
        else if (i < text.length && text[i] !== pattern[j]) {
            comparisons++;
            if (j !== 0) {
                j = lps[j - 1];
            }
            else {
                i++;
            }
        }
    }
    return { indexes: result, comparisons };
}

export function searchKMP(text: string, keywords: string[]):
AlgorithmResult {
    const startTime = performance.now();
    const matches: MatchResult[] = [];
    let totalComparisons = 0;
    const lowerText = text.toLowerCase();
    for (const keyword of keywords) {
        const cleanKeyword = keyword.trim();
```

```

    if (cleanKeyword.length === 0) {
      continue;
    }
    const lowerKeyword = cleanKeyword.toLowerCase();
    const result = kmpSearch(lowerText, lowerKeyword);
    totalComparisons += result.comparisons;
    for (const startIndex of result.indexes) {
      const endIndex = startIndex + cleanKeyword.length;
      matches.push({
        keyword: cleanKeyword,
        matchedText: text.slice(startIndex, endIndex),
        startIndex,
        endIndex,
        algorithm: "KMP",
        similarity: 1,
      });
    }
  }
  return {
    algorithm: "KMP",
    matches,
    stats: {
      algorithm: "KMP",
      totalMatches: matches.length,
      comparisons: totalComparisons,
      executionTimeMs: performance.now() - startTime,
    },
  },
};

```

4.3.2. Boyer-Moore

Fungsi utama `searchBoyerMoore(text, keywords)` bekerja serupa dengan KMP — iterasi per kata kunci, normalisasi ke huruf kecil, lalu memanggil `boyerMooreSearch`. Boyer-Moore berbeda fundamental karena mencocokkan pattern dari kanan ke kiri.

Fungsi `buildLastOccurrenceTable(pattern)` membangun sebuah Map yang menyimpan indeks terakhir kemunculan setiap karakter dalam pattern:

Tabel ini digunakan untuk menentukan seberapa jauh window pencarian dapat digeser saat terjadi mismatch. Jika karakter tidak ada dalam tabel, nilai default -1 digunakan.

Beberapa komponen penting dalam implementasi algoritma ini adalah sebagai berikut :

- `patternIndex` adalah posisi mismatch di pattern (dihitung dari kanan).
- `lastIndex` adalah posisi terakhir karakter tersebut dalam pattern.

- $\text{move} = \text{patternIndex} - \text{lastIndex} \rightarrow$ seberapa jauh pattern digeser agar karakter yang salah di teks sejajar dengan kemunculan terakhirnya di pattern.
- $\text{Math.max}(1, \text{move})$ memastikan shift selalu maju minimal 1, mencegah infinite loop.

Saat match ditemukan ($\text{patternIndex} < 0$), posisi dicatat dan shift maju 1. Pada kasus terbaik, kompleksitas mencapai $O(n/m)$.

Perbandingan dihitung hanya selama iterasi pencocokan karakter di loop while ($\text{patternIndex} \geq 0$). Setiap kali karakter pattern dibandingkan dengan karakter teks, nilai dari variabel comparisons akan diincrement dipanggil. Ini mencerminkan sifat Boyer-Moore yang melakukan lebih sedikit perbandingan dibanding KMP pada data nyata karena sering melewati banyak karakter sekaligus.

```
function boyerMooreSearch(text: string, pattern: string): { indexes:
number[]; comparisons: number } {
  if (pattern.length === 0 || text.length === 0 || pattern.length >
text.length) {
    return { indexes: [], comparisons: 0 };
  }

  const lastOccurrence = buildLastOccurrenceTable(pattern);
  const result: number[] = [];
  let comparisons = 0;
  let shift = 0;
  while (shift <= text.length - pattern.length) {
    let patternIndex = pattern.length - 1;
    while (patternIndex >= 0) {
      comparisons++;
      if (pattern[patternIndex] !== text[shift + patternIndex])
      {
        break;
      }
      patternIndex--;
    }
    if (patternIndex < 0) {
      result.push(shift);
      shift += 1;
    } else {
      const mismatchedChar = text[shift + patternIndex];
      const lastIndex = lastOccurrence.get(mismatchedChar) ??
-1;

      const move = patternIndex - lastIndex;
      shift += Math.max(1, move);
    }
  }
}
```

```

    }
    return {indexes: result, comparisons};
}

export function searchBoyerMoore(text: string, keywords: string[]):
AlgorithmResult {
    const startTime = performance.now();
    const matches: MatchResult[] = [];
    let totalComparisons = 0;
    const lowerText = text.toLowerCase();
    for (const keyword of keywords) {
        const cleanKeyword = keyword.trim();
        if (cleanKeyword.length === 0) {
            continue;
        }
        const lowerKeyword = cleanKeyword.toLowerCase();
        const result = boyerMooreSearch(lowerText, lowerKeyword);
        totalComparisons += result.comparisons;
        for (const startIndex of result.indexes) {
            const endIndex = startIndex + cleanKeyword.length;
            matches.push({
                keyword: cleanKeyword,
                matchedText: text.slice(startIndex, endIndex),
                startIndex,
                endIndex,
                algorithm: "Boyer-Moore",
                similarity: 1,
            });
        }
    }
    return {
        algorithm: "Boyer-Moore",
        matches,
        stats: {
            algorithm: "Boyer-Moore",
            totalMatches: matches.length,
            comparisons: totalComparisons,
            executionTimeMs: performance.now() - startTime,
        },
    };
}

```

4.3.3. Regular Expression

Regular Expression digunakan untuk mendeteksi kata - kata mencurigakan yang memiliki pola <teks><angka>. Dalam hal ini, token/kata hanya dianggap mencurigakan

apabila minimal mengandung 3 huruf dan 2 angka secara bersamaan. Filter ini menghilangkan false positive seperti token yang terlalu pendek atau yang kebetulan mengandung angka namun bukan pola yang relevan.

Berikut adalah implementasi dari algoritma Regular Expression yang digunakan dalam bahasa *Typescript*.

```
export function searchRegex(text: string): AlgorithmResult {
  const startTime = performance.now();
  const matches: MatchResult[] = [];
  WORD_WITH_NUMBER_PATTERN.lastIndex = 0;
  for (const match of text.matchAll(WORD_WITH_NUMBER_PATTERN)) {
    const matchedText = match[0];
    const startIndex = match.index ?? 0;
    const endIndex = startIndex + matchedText.length;
    if (!isSuspiciousToken(matchedText)) {
      continue;
    }
    matches.push({
      keyword: matchedText,
      matchedText,
      startIndex,
      endIndex,
      algorithm: "Regex",
      similarity: 1,
    });
  }
  return {
    algorithm: "Regex",
    matches,
    stats: {
      algorithm: "Regex",
      totalMatches: matches.length,
      executionTimeMs: performance.now() - startTime,
    },
  };
}
```

4.3.4. Weighted Levenshtein Distance

Pertama - tama, kita akan menjalankan dengan menggunakan *dynamic programming* 2 state. Ukuran dari dp ini adalah $(|source| + 1) \times (|target| + 1)$. Pada awalnya, kita menginisialisasi $dp[i][0] = i$ untuk semua baris i ke 0 hingga baris i ke -rows, dan $dp[0][j]$ untuk setiap kolom j ke 0 hingga kolom j ke -rows. Transisi utama dari dp yang

digunakan adalah $dp[i][j] = \min(dp[i-1][j] + 1, dp[i][j-1] + 1, dp[i-1][j-1] + \text{substitutionCost}(\text{source}[i-1], \text{target}[j-1]))$. Nilai akhir $dp[\text{source}][\text{target}]$ adalah jarak edit antara kedua string.

Berbeda dari Levenshtein standar (semua substitusi berbobot 1), implementasi ini memberikan biaya lebih rendah untuk pasangan karakter yang secara visual atau fonetis mirip.

Fungsi `calculateSimilarity()` menghitung skor kemiripan pada rentang [0, 1]. Token dari teks hanya dibandingkan jika lolos filter `isReasonableLength`, yang membatasi perbedaan panjang maksimal sebesar $\max(2, \lfloor |\text{keyword}| \times 0.4 \rfloor)$. Ini menghindari perbandingan yang jelas tidak relevan dan menghemat waktu komputasi.

Berikut adalah implementasi dari weighted Levenshtein Distance dalam bahasa *Typescript*.

```
export function searchWeightedLevenshtein(
  text: string,
  keywords: string[],
  threshold = DEFAULT_SIMILARITY_THRESHOLD
): AlgorithmResult {
  const startTime = performance.now();
  const matches: MatchResult[] = [];
  const tokens = getCandidateTokens(text);

  for (const { token, startIndex, endIndex } of tokens) {
    let bestKeyword = "";
    let bestSimilarity = 0;
    for (const keyword of keywords) {
      const cleanKeyword = keyword.trim();
      if (cleanKeyword.length === 0 ||
!isReasonableLength(token, cleanKeyword)) {
        continue;
      }
      const similarity = calculateSimilarity(token,
cleanKeyword);

      if (similarity > bestSimilarity) {
        bestKeyword = cleanKeyword;
        bestSimilarity = similarity;
      }
    }
  }
}
```

```

        if (bestSimilarity >= threshold && token.toLowerCase() !==
bestKeyword.toLowerCase()) {
            matches.push({
                keyword: bestKeyword,
                matchedText: token,
                startIndex,
                endIndex,
                algorithm: "Weighted-Levenshtein",
                similarity: bestSimilarity,
            });
        }
    }

    return {
        algorithm: "Weighted-Levenshtein",
        matches,
        stats: {
            algorithm: "Weighted-Levenshtein",
            totalMatches: matches.length,
            executionTimeMs: performance.now() - startTime,
        },
    };
}

```

4.3.5. Aho-Corasick

Perhatikan bahwa struktur data utama yang digunakan dalam algoritma ini adalah trie.

Aho-Corasick menggunakan AhoCorasickNode yang memiliki tiga komponen:

- children: Map<string, AhoCorasickNode>.Transisi karakter ke node berikutnya.
- failureLink: AhoCorasickNode | null. Pointer ke node terpanjang yang merupakan proper suffix dari path saat ini.
- output: string[]. kata kunci yang berakhir di node ini (termasuk dari failure link).

Semua kata kunci dimasukkan ke dalam Trie terlebih dahulu via buildTrie. Kemudian buildFailureLinks menggunakan BFS untuk membangun failure links di seluruh node. Output setiap node juga digabungkan dengan output dari failure link-nya, memungkinkan deteksi overlap kata kunci.

Pencarian hanya memerlukan satu kali traversal teks untuk seluruh kata kunci sekaligus:

Kompleksitas total dari algoritma ini adalah $O(n + m + z)$ dimana n = panjang teks, m = total panjang semua keyword, z = jumlah match. Ini menjadikan Aho-Corasick jauh lebih efisien dari KMP atau Boyer-Moore ketika jumlah kata kunci sangat besar.

```
class AhoCorasick {
  private root: AhoCorasickNode = new AhoCorasickNode();

  constructor(keywords: string[]) {
    this.buildTrie(keywords);
    this.buildFailureLinks();
  }

  private buildTrie(keywords: string[]) {
    for (const keyword of keywords) {
      const cleanKeyword = keyword.trim();
      if (cleanKeyword.length === 0) continue;

      let current = this.root;
      const lowerKeyword = cleanKeyword.toLowerCase();

      for (let i = 0; i < lowerKeyword.length; i++) {
        const char = lowerKeyword[i];
        if (!current.children.has(char)) {
          current.children.set(char, new
AhoCorasickNode());
        }
        current = current.children.get(char)!;
      }
      current.output.push(cleanKeyword);
    }
  }

  private buildFailureLinks() {
    const queue: AhoCorasickNode[] = [];

    for (const [, childNode] of this.root.children) {
      childNode.failureLink = this.root;
      queue.push(childNode);
    }

    while (queue.length > 0) {
      const current = queue.shift();

      for (const [char, childNode] of current.children) {

```

```

        let failure = current.failureLink;

        while (failure !== null &&
!failure.children.has(char)) {
            failure = failure.failureLink;
        }

        if (failure === null) {
            childNode.failureLink = this.root;
        } else {
            childNode.failureLink =
failure.children.get(char)!;
        }

        if (childNode.failureLink.output.length > 0) {
            childNode.output =
childNode.output.concat(childNode.failureLink.output);
        }

        queue.push(childNode);
    }
}

public search(text: string): {
    matches: { keyword: string; startIndex: number; endIndex:
number }[];
    comparisons: number;
}
{
    const results: { keyword: string; startIndex: number;
endIndex: number }[] = [];
    let comparisons = 0;
    let current = this.root;
    const lowerText = text.toLowerCase();

    for (let i = 0; i < lowerText.length; i++) {
        const char = lowerText[i];

        while (current !== this.root &&
!current.children.has(char)) {
            comparisons++;
            current = current.failureLink!;
        }
    }
}

```

```

        comparisons++;
        if (current.children.has(char)) {
            current = current.children.get(char)!;
        } else {
            current = this.root;
        }

        if (current.output.length > 0) {
            for (const keyword of current.output) {
                const len = keyword.length;
                const startIndex = i - len + 1;
                results.push({
                    keyword,
                    startIndex,
                    endIndex: i + 1,
                });
            }
        }
    }

    return { matches: results, comparisons };
}

export function searchAhoCorasick(text: string, keywords: string[]):
AlgorithmResult {
    const startTime = performance.now();
    const matches: MatchResult[] = [];
    const ac = new AhoCorasick(keywords);
    const { matches: rawMatches, comparisons } = ac.search(text);

    for (const raw of rawMatches) {
        matches.push({
            keyword: raw.keyword,
            matchedText: text.slice(raw.startIndex, raw.endIndex),
            startIndex: raw.startIndex,
            endIndex: raw.endIndex,
            algorithm: "Aho-Corasick",
            similarity: 1,
        });
    }

    return {
        algorithm: "Aho-Corasick",
        matches,
    };
}

```

```

    stats: {
      algorithm: "Aho-Corasick",
      totalMatches: matches.length,
      comparisons,
      executionTimeMs: performance.now() - startTime,
    },
  };
}

```

4.3.6. Rabin-Karp

Algoritma Rabin-Karp menggunakan fungsi hash polinomial untuk menghindari perbandingan karakter-per-karakter pada setiap posisi:

Nilai $h = d^{(m-1)} \% q$ dihitung di awal dan digunakan untuk memperbarui hash window dengan $O(1)$. Penanganan $t < 0$ penting karena operasi pengurangan dalam modular aritmetika dapat menghasilkan nilai negatif di JavaScript.

Hash yang sama (hash collision) tidak langsung dianggap match. Verifikasi karakter dilakukan hanya ketika $p === t$:

Oleh karena itu, comparisons hanya dihitung saat verifikasi karakter berlangsung, bukan pada setiap pergeseran window. Pada kasus ideal tanpa hash collision, kompleksitas menjadi $O(n + m)$. Pemilihan bilangan prima $q = 101$ adalah salah satu pendekatan yang baik. Bilangan prima 101 cukup kecil untuk menghindari integer overflow, tetapi cukup besar juga untuk meminimalkan probabilitas terjadinya false collision.

```

function rabinKarpSearch(text: string, pattern: string): { indexes:
number[]; comparisons: number } {
  const n = text.length;
  const m = pattern.length;

  if (m === 0 || n < m) {
    return { indexes: [], comparisons: 0 };
  }
  const d = 256;
  const q = 101;
  let h = 1;
  let p = 0;
  let t = 0;
  let comparisons = 0;
  const indexes: number[] = [];
  for (let i = 0; i < m - 1; i++) {
    h = (h * d) % q;
  }
  for (let i = 0; i < m; i++) {

```

```

        p = (d * p + pattern.charCodeAt(i)) % q;
        t = (d * t + text.charCodeAt(i)) % q;
    }
    for (let i = 0; i <= n - m; i++) {
        if (p === t) {
            let match = true;
            for (let j = 0; j < m; j++) {
                comparisons++;
                if (text[i + j] !== pattern[j]) {
                    match = false;
                    break;
                }
            }
            if (match) {
                indexes.push(i);
            }
        }
        if (i < n - m) {
            t = (d * (t - text.charCodeAt(i) * h) + text.charCodeAt(i
+ m)) % q;
            if (t < 0) {
                t = t + q;
            }
        }
    }
    return { indexes, comparisons };
}

export function searchRabinKarp(text: string, keywords: string[]):
AlgorithmResult {
    const startTime = performance.now();
    const matches: MatchResult[] = [];
    let totalComparisons = 0;
    const lowerText = text.toLowerCase();

    for (const keyword of keywords) {
        const cleanKeyword = keyword.trim();
        if (cleanKeyword.length === 0) {
            continue;
        }

        const lowerKeyword = cleanKeyword.toLowerCase();
        const result = rabinKarpSearch(lowerText, lowerKeyword);
        totalComparisons += result.comparisons;
    }
}

```

```

    for (const startIndex of result.indexes) {
      const endIndex = startIndex + cleanKeyword.length;
      matches.push({
        keyword: cleanKeyword,
        matchedText: text.slice(startIndex, endIndex),
        startIndex,
        endIndex,
        algorithm: "Rabin-Karp",
        similarity: 1,
      });
    }
  }

  return {
    algorithm: "Rabin-Karp",
    matches,
    stats: {
      algorithm: "Rabin-Karp",
      totalMatches: matches.length,
      comparisons: totalComparisons,
      executionTimeMs: performance.now() - startTime,
    },
  };
}

```

4.4. Implementasi Browser Extension

4.4.1. Manifest.json

Seluruh ekstensi dimulai dari manifest.json yang bertindak sebagai kontrak antara kode dan browser. Ekstensi ini menggunakan Manifest V3, yang mewajibkan penggunaan service worker sebagai background context. Deklarasi `permissions: ["storage", "activeTab"]` memberi izin akses ke `chrome.storage.local` untuk menyimpan statistik dan settings, serta akses ke tab aktif agar popup bisa mengirim pesan rescan. `host_permissions: ["<all_urls>"]` diperlukan agar background worker bisa melakukan fetch gambar lintas domain. Bagian `web_accessible_resources` mendaftarkan file Tesseract worker dan WASM core agar bisa diakses oleh content script melalui `chrome.runtime.getURL()`

4.4.2. Background Service Worker

Background service worker di `background.ts` berperan sebagai proxy untuk dua operasi yang tidak bisa dilakukan content script secara langsung. Pertama, fetch gambar lintas domain: content script tidak bisa memanggil `fetch()` ke URL dari domain lain karena pembatasan CORS di halaman web, namun

background worker memiliki akses penuh. Ketika content script mengirim pesan SWEETBONANZA_FETCH_IMAGE beserta URL-nya, background menjalankan fetchImageAsDataURL() yang mengambil gambar, mengkonversi buffer ke base64 dengan arrayBufferToBase64(), lalu mengirim data URL kembali ke content script. Kedua, pengambilan screenshot tab aktif melalui chrome.tabs.captureVisibleTab()/ Saat content script mengirim SWEETBONANZA_CAPTURE_VISIBLE_TAB, background mengambil screenshot dalam format PNG dan mengirimnya balik. Pola return true di dalam onMessage.addListener penting karena memberitahu Chrome bahwa response akan dikirim secara asinkron, sehingga koneksi messaging tidak ditutup sebelum promise selesai.

4.4.3. Content Script & Pemindaian DOM

Content script di content.ts adalah inti dari seluruh ekstensi. Segera setelah dimuat, ia memanggil performScan() yang memulai siklus pemindaian. Fungsi ini pertama-tama memanggil clearHighlights() untuk menghapus semua elemen <mark> dari sesi sebelumnya dengan cara mengganti setiap mark kembali menjadi text node dan memanggil normalize() pada parent-nya agar text node yang berdekatan digabung kembali. Selanjutnya resetStatistics() mengosongkan semua Map yang menyimpan frekuensi kata, waktu eksekusi, jumlah match, dan jumlah perbandingan per algoritma. Setelah state bersih, highlightMatches() dipanggil.

Di dalam highlightMatches(), sebuah TreeWalker menelusuri seluruh text node di document.body, melewati elemen script, style, noscript, mark, input, textarea, code, pre, serta elemen contentEditable agar highlight tidak muncul di tempat yang tidak seharusnya. Setiap text node yang lolos filter dikumpulkan dalam array, lalu masing-masing dijalankan melalui enam algoritma string matching secara bersamaan: Boyer-Moore, KMP, Regex, Weighted Levenshtein, Aho-Corasick, dan Rabin-Karp. Semua hasil digabung di handleMultipleAlgorithms(), diurutkan berdasarkan posisi, dikelompokkan jika ada match yang tumpang tindih di posisi yang sama, lalu difilter untuk menghilangkan overlap. Dari sini dibangun DocumentFragment yang memisahkan teks biasa dan teks yang cocok, di mana teks yang cocok dibungkus dalam elemen <mark> berwarna merah dengan teks putih.

4.4.4. Highlight dan Tooltip

Setiap elemen <mark> yang dibuat menyimpan informasi penting di atribut dataset: nama keyword yang cocok (data-keyword) dan daftar algoritma yang mendeteksinya (data-algorithms). Tooltip dibuat sebagai satu elemen <div> tunggal yang diposisikan secara absolut di document.body, sehingga lebih efisien

dalam penggunaan memori DOM. Event listener mouseenter pada setiap mark membaca dataset tersebut, mengambil frekuensi dari wordFrequencies Map, dan mengisi innerHTML tooltip dengan informasi keyword, algoritma, frekuensi kemunculan, serta waktu eksekusi masing-masing algoritma. Event mousemove memperbarui posisi tooltip mengikuti kursor, sementara mouseleave menyembunyikannya.

4.4.5. MutationObserver dan Rescan

Untuk menangani halaman yang memuat konten secara dinamis seperti infinite scroll atau konten yang di-render via JavaScript, content script mengaktifkan MutationObserver setelah setiap scan selesai. Observer ini memantau perubahan childList dan characterData di seluruh subtree document.body. Ketika perubahan terdeteksi, ia memeriksa apakah mutasi tersebut bukan dari highlight yang baru saja dibuat sendiri lalu menjadwalkan ulang performScan() dengan debounce 500ms. Observer dihentikan saat scan sedang berjalan dan diaktifkan kembali di blok finally. Selain dari MutationObserver, rescan juga bisa dipicu secara manual melalui popup: popup mengirim pesan SWEETBONANZA_RESCAN via chrome.tabs.sendMessage ke tab aktif, content script menangkapnya di chrome.runtime.onMessage dan memanggil performScan() kembali.

4.4.6. OCR Gambar

Fitur OCR memungkinkan ekstensi mendeteksi keyword yang muncul di dalam gambar, bukan hanya teks DOM. Ketika OCR diaktifkan, scanImagesWithOcr() dijalankan. Pertama ia menginisialisasi Tesseract.js worker secara singleton melalui getOcrWorker(). Sebelum memproses gambar individual, fungsi ini terlebih dahulu mencoba OCR screenshot tab aktif melalui scanVisibleTabScreenshotWithOcr(). Setiap posisi viewport (dihitung dari scrollY / innerHeight) hanya diproses sekali per sesi scan menggunakan processedScreenshotSegments Set, sehingga scroll tidak menyebabkan pemrosesan berulang.

Untuk gambar individual, getOcrCandidateImages() menyaring elemen yang visible, berukuran minimal 80×80 piksel, belum pernah diproses, dan memiliki kurang dari dua kegagalan sebelumnya. Setiap gambar kemudian melalui preprocessImageDataUrl() yang menggambar ulang gambar di <canvas> dengan filter grayscale(1) contrast(1.35), serta scaling ke rentang 900–1600 piksel agar akurasi OCR optimal. Hasil teks dari Tesseract dijalankan kembali melalui keenam algoritma string matching lewat findOcrTextMatches(). Jika ada keyword yang ditemukan, applyOcrImageEffect() menerapkan filter: blur(8px) dan outline merah pada elemen tersebut, serta menyimpan kata-kata yang terdeteksi di

dataset. Setiap operasi OCR dilindungi timeout (45 detik per gambar, 60 detik untuk inisialisasi worker) menggunakan `Promise.race` di `withTimeout()`.

4.4.7. Penyimpanan Statistik

`statsStore.ts` mengelola persistensi statistik scan ke `chrome.storage.local` dengan key `sweetbonanza.scanStatistics`. `writeScanStatistics()` dipanggil dari content script setiap kali scan selesai atau setiap kali hasil OCR baru masuk. Yang paling menarik adalah `subscribeScanStatistics()` yang memanfaatkan `chrome.storage.onChanged`. Ini berarti popup yang sedang terbuka otomatis menerima statistik terbaru secara real-time tanpa perlu polling, karena content script menulis ke storage dan popup langsung mendapat notifikasi perubahan.

4.4.8. Settings Store dan Blur Teks

`settingsStore.ts` menggunakan pola yang sama untuk dua boolean setting, yaitu `ocrEnabled` dan `blurEnabled`. `subscribeOcrEnabled()` memungkinkan content script bereaksi langsung saat pengguna mengubah toggle di popup: jika OCR diaktifkan, `scheduleOcrScan(0)` dipanggil segera; jika dinonaktifkan, `clearOcrImageEffects()` mengembalikan semua gambar yang ter-blur ke kondisi normal dan `resetOcrStatistics()` membersihkan statistik OCR dari total. `subscribeBlurEnabled()` memicu `applyTextBlur()` yang mengiterasi semua elemen `.sweetbonanza-highlighted-word` yang ada di halaman dan menambahkan atau menghapus filter: `blur(4px)`. State `currentBlurEnabled` juga dibaca saat highlight baru dibuat di `handleMultipleAlgorithms()`, sehingga keyword yang ditemukan saat blur sedang aktif langsung ter-blur tanpa perlu apply ulang setelahnya.

4.4.9. Popup

`popup.ts` me-render antarmuka statistik yang terdiri dari tiga bagian: metadata scan (judul halaman, domain, dan waktu scan), statistik per algoritma dengan bar visualisasi proportional terhadap jumlah match terbanyak, dan frekuensi keyword yang diurutkan dari yang paling sering muncul. Semua elemen DOM dibuat secara programatik menggunakan `document.createElement` tanpa template HTML. Popup juga mengelola toggle OCR dan blur: perubahan di toggle langsung ditulis ke storage via `writeOcrEnabled()` atau `writeBlurEnabled()`, dan jika penulisan gagal, toggle dikembalikan ke state sebelumnya sebagai mekanisme rollback. Seluruh alur komunikasi antara popup, content script, background worker, dan storage tersambung menjadi satu sistem yang bekerja secara reaktif.

4.5. Pengujian

4.5.1. Hasil Pengujian

Pengujian 1: Hasil query google search “gacor” (untuk tes hasil dan statistik popup)

The image displays a Google search interface with the query "gacor". The search results are shown on the left, and a sidebar titled "Judol Detector" is on the right. The sidebar provides statistics for the keyword "gacor" and lists various algorithms used for keyword detection.

Google Search Results (Top Snippets):

- Princeton University: 12NAGA: Terpercaya & Online Gampang ...
- seungmopark.com: RUMAH TOTO Link Situs Toto Server ...
- unej.ac.id: 88: Agen Resmi 88 Top 1 Asia dengan Super ...
- Universitas Slamet Riyadi: Daftar Thailand Resmi Anti Rungkat ...
- migrantsandmembership.org: Official Situs Anti Rungkat ...
- theedgewoodontheedge.com

Judol Detector Statistics:

- Total Keyword Ditemukan:** 106
- Scan Terakhir:** HALAMAN: gacor - Penelusuran Google; DOMAIN: www.google.com; WAKTU: 30 Mei 2026, 20.02
- Statistik Algoritma:**
 - Boyer-Moore:** 155 match; WAKTU: 92.20 ms; COMPARISON: 322751
 - KMP:** 155 match; WAKTU: 84.70 ms; COMPARISON: 8065442
 - Regex:** 11 match; WAKTU: 2.20 ms; COMPARISON: 0
 - Weighted-Levenshtein:** 1 match; WAKTU: 2792.60 ms; COMPARISON: 0
 - Aho-Corasick:** 155 match; WAKTU: 268.30 ms; COMPARISON: 12289
 - Rabin-Karp:** 155 match; WAKTU: 95.50 ms; COMPARISON: 23199
 - OCR:** 0 match; WAKTU: 0.00 ms; INPUT: 0
- Perbandingan Keyword:**

Google

gacor

Mode AI Semua Video Gambar Shopping Berita Video singkat Lainnya Alat

Princeton University

https://peet.princeton.edu

12NAGA: Terpercaya & Online Gampang ...

12NAGA adalah resmi yang menyediakan game online gampang menang dan 777 terpercaya. Daftarkan diri Anda untuk meraih ...

seungmopark.com

https://thai.seungmopark.com

RUMAH TOTO Link Situs Toto Server ...

RUMAH TOTO hadir sebagai link situs toto yang dirancang dengan sistem akses lebih stabil, ringan dibuka di berbagai perangkat, ...

tnj.ac.id

https://informasi.tnj.ac.id/Sibolga-Julu

88: Agen Resmi 88 Top 1 Asia dengan Super ...

Universitas Slamet Riyadi

https://map.fkip.unsri.ac.id

Daftar Thailand Resmi Anti Rungkat ...

Daftar Thailand Resmi adalah platform permainan online terpercaya yang menghadirkan asli, permainan anti rungkat, ...

migrantsandmembership.org

https://ngom.migrantsandmembership.org

Official Situs Anti Rungkat

adalah resmi dengan kemenangan maksimal dan fitur terlengkap dengan berbagai banyak promo dan setiap saat.

theedgewoodthegreen.com

Rabin-Karp

155 match

WAKTU 95.50 ms

COMPARISON 23199

OCR

1 match

WAKTU 3595.80 ms

INPUT 1

Perbandingan Keyword

slot 43

gacor 33

situs slot 9

Rescan Halaman

OCR gambar

OCR aktif. Gambar visible akan diproses.

Blur teks

Blur teks aktif untuk hasil deteksi.

Pengujian 2: Hasil query google search “merdeka 45” (untuk tes Regex)

Merdeka 45 News

https://news.com

Merdeka 45 News

News| Karawang – Program Strategis Nasional Koperasi Desa/Kelurahan Merah Putih resmi diluncurkan secara serentak di 1.061 titik di seluruh Indonesia ...

88

https://nm

Keyword: Merdeka45

Algoritma: Regex

Frekuensi: 17 kali

Total Waktu Eksekusi: 5

- Regex: 2.400 ms

line Pelayanan Terbaik 24 Jam ...

memili a melayani para member selama 24 jam penuh dengan moto

keramahan situs g

detik.com

https://www.detik.com

detik.com

Home · Nasional · Daerah · Ekonomi · Sosial Budaya · Pemkab Murung Raya · DPRD Murung Raya; usdc; shiba-inu; avalanche; pi-network; hyperliquid; monero ...

Google

merdeka 45

Mode AI Semua Gambar Video Berita Shopping Maps Lainnya Alat

Merdeka 45 News

https://news.com

Merdeka 45 News

News Karawang – Program Strategis Nasional Koperasi Desa/Kelurahan Merah Putih resmi diluncurkan secara serentak di 1.061 titik di seluruh Indonesia ...

Agensi Game Online Pelayanan Terbaik 24 Jam ...

memiliki Admin CS yang setia melayani para member selama 24 jam penuh dengan motto keramahan situs game online Merdeka 45.

detik.com

https://www.detik.com

detik.com

Home Nasional Daerah Ekonomi Sosial Budaya Pemkab Murung Raya DPRD Murung Raya, usdc, shiba-inu, avalanche, pi-network, hyperliquid, monero ...

Ringkasan AI

3

"Merdeka 45" paling sering merujuk pada lagu nasional "17 Agustus Tahun 45" atau lokasi bersejarah seperti Gedung Merdeka dan Museum Joang 45. YouTube sitasitu +1

Detail utama mengenai referensi populer tersebut meliputi:

Lagu Hari Merdeka (17 Agustus 1945): Lagu wajib nasional Indonesia ciptaan H. Mutahar yang selalu dinyanyikan saat peringatan HUT Kemerdekaan RI. YouTube sitasitu +1

Gedung Merdeka: Berlokasi di Jalan Asia-Afrika No. 65, Kota Bandung, ini adalah bangunan bersejarah bergaya Art Deco yang menjadi lokasi utama Konferensi Asia-

SWEETBONANZA

Judol Detector

Statistik scan terbaru diterima.

Total Keyword Ditemukan

17

Scan Terakhir

HALAMAN

merdeka 45 - Penelusuran Google

DOMAIN

www.google.com

WAKTU

30 Mei 2026, 20.22

Statistik Algoritma

Boyer-Moore

WAKTU

68.00 ms

COMPARISON

416209

0 match

KMP

WAKTU

COMPARISON

0 match

KMP

WAKTU

95.00 ms

COMPARISON

10093092

0 match

Regex

WAKTU

2.60 ms

COMPARISON

0

17 match

Weighted-Levenshtein

WAKTU

3572.00 ms

COMPARISON

0

0 match

Aho-Corasick

WAKTU

264.00 ms

COMPARISON

16321

0 match

Rabin-Karp

WAKTU

122.40 ms

COMPARISON

30878

0 match

OCR

WAKTU

8502.00 ms

INPUT

6

0 match

Perbandingan Keyword

46

Google

merdeka 45

Mode AI Semua Gambar Video Berita Shopping Maps Lainnya Alat

Merdeka 45 News

https://news.com

Merdeka 45 News

News Karawang – Program Strategis Nasional Koperasi Desa/Kelurahan Merah Putih resmi diluncurkan secara serentak di 1.061 titik di seluruh Indonesia ...

HA

https://org

: Agen Game Online Pelayanan Terbaik 24 Jam ...

memiliki Admin CS yang setia melayani para member selama 24 jam penuh dengan moto keramahan situs game online Merdeka 45.

detik.com

https://www.detik.com

detik.com

Home Nasional Daerah Ekonomi Sosial Budaya Pemkab Murung Raya DPRD Murung Raya usdc shiba-inu avalanche pi-network hyperliquid monero ...

Ringkasan AI

"Merdeka 45" paling sering merujuk pada lagu nasional "17 Agustus Tahun 45" atau lokasi bersejarah seperti Gedung Merdeka dan Museum Joang 45. YouTube situsitu 41

Detail utama mengenai referensi populer tersebut meliputi:

- Lagu Hari Merdeka (17 Agustus 1945):** Lagu wajib nasional Indonesia ciptaan H. Mutahar yang selalu dinyanyikan saat peringatan HUT Kemerdekaan RI. YouTube situsitu 41
- Gedung Merdeka:** Berlokasi di Jalan Asia-Afrika No. 65, Kota Bandung, ini adalah bangunan bersejarah bergaya Art Deco yang menjadi lokasi utama Konferensi Asia-

3572.00 ms 0

Aho-Corasick

0 match

WAKTU 264.00 ms

COMPARISON 16321

Rabin-Karp

0 match

WAKTU 122.40 ms

COMPARISON 30878

OCR

0 match

WAKTU 8502.00 ms

INPUT 6

Perbandingan Keyword

merdeka45 17

Rescan Halaman

☒ OCR gambar

OCR aktif. Gambar visible akan diproses.

☒ Blur teks

Blur teks aktif untuk hasil deteksi.

Pengujian 3: <https://hantu-unjukgigi.pages.dev/> (untuk tes ocr dan blur gambar)

UNSURHOKI

Selamat Datang di UNSURHOKI RAJA GA

DOWNLOAD APK UNSURHOKI

Sekarang Juga!!

FREEBET RP.5000

KLAIM MELALUI CS UNSURHOKI

LOGIN RESMI

DAFTAR AKUN VVIP

LAYANAN CS 24 JAM

SITUS GACOR

HANTULOKAL @Copyright 2025 UNSURHOKI

47.00 ms 0

Aho-Corasick

1 match

WAKTU 7.70 ms

COMPARISON 179

Rabin-Karp

1 match

WAKTU 1.80 ms

COMPARISON 202

OCR

0 match

WAKTU 0.00 ms

INPUT 0

Perbandingan Keyword

gacor 1

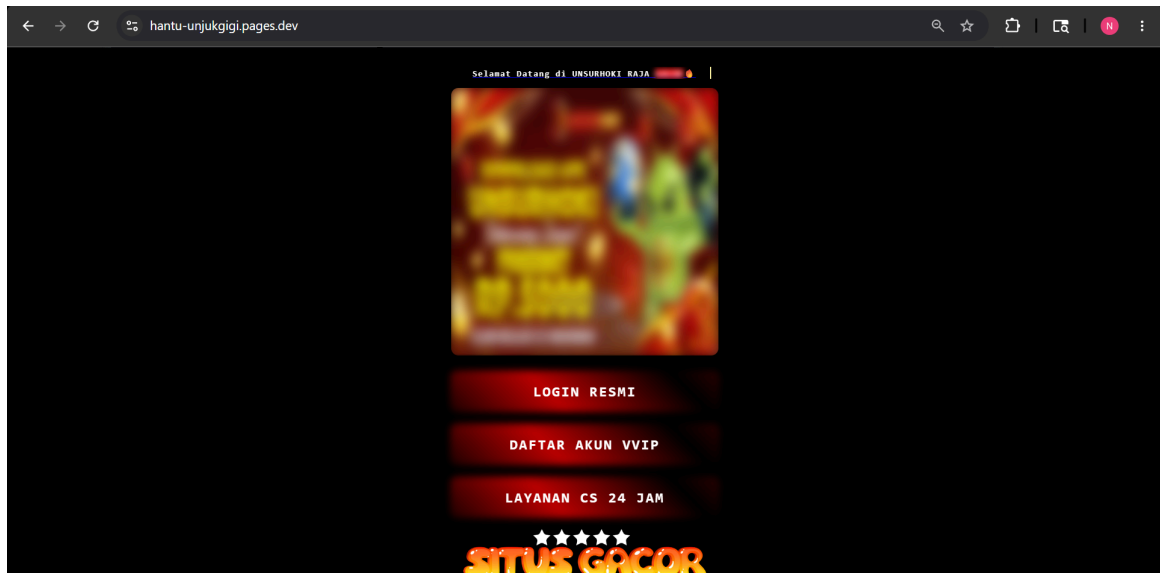
Rescan Halaman

☐ OCR gambar

OCR nonaktif.

☐ Blur teks

Blur teks nonaktif.



Selamat Datang di UNSURHOKI RAJA

LOGIN RESMI

DAFTAR AKUN VVIP

LAYANAN CS 24 JAM

SITUS GACOR

Total Keyword Ditemukan

2

Scan Terakhir

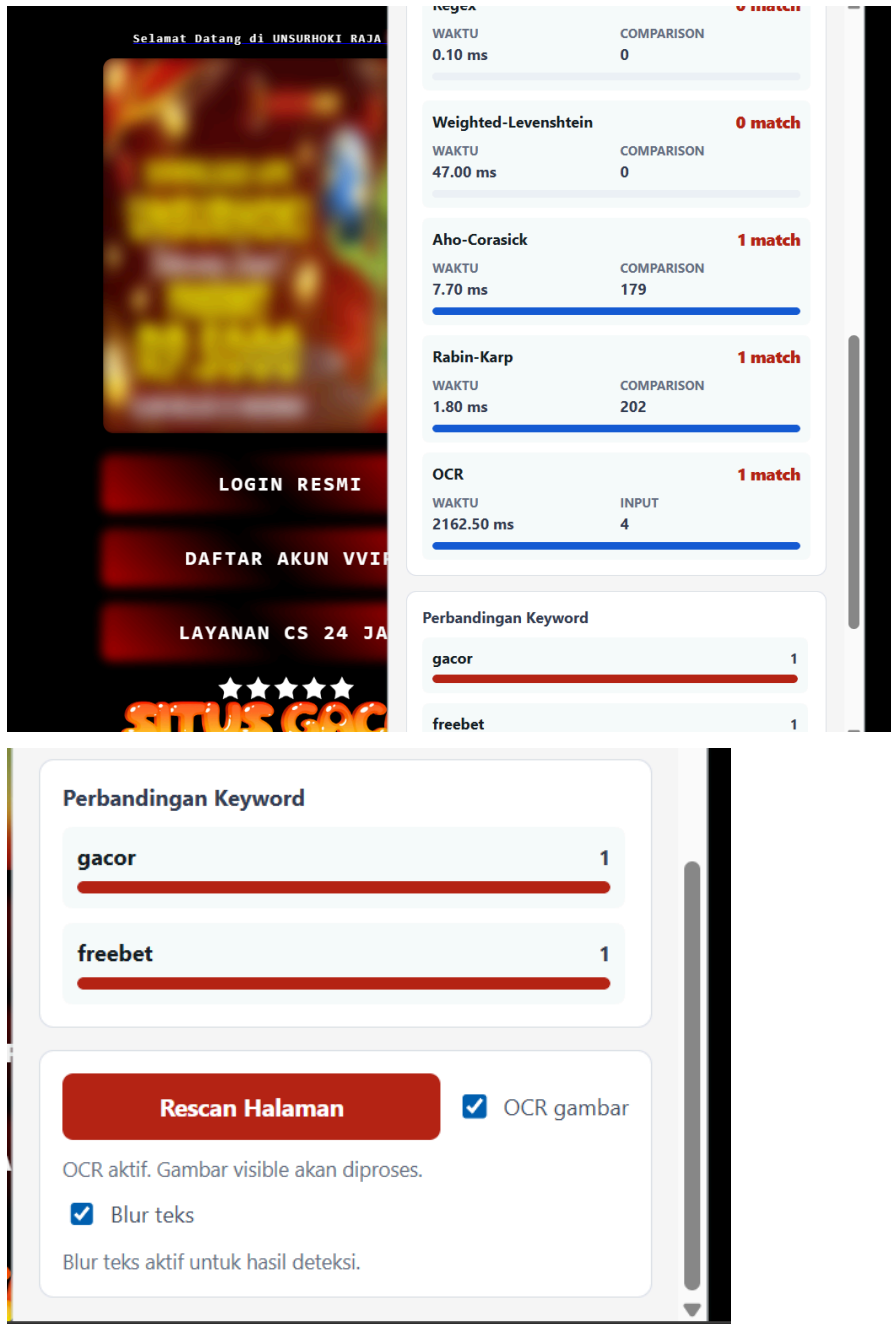
HALAMAN
HOKI777 : Situs Link Slot Gacor 777 Maxwin Bet ...

DOMAIN
hantu-unjukgigi.pages.dev

WAKTU
30 Mei 2026, 20.49

Statistik Algoritma

Boyer-Moore		1 match
WAKTU	COMPARISON	
2.20 ms	3807	
KMP		1 match
WAKTU	COMPARISON	
2.40 ms	138302	
Regex		0 match
WAKTU	COMPARISON	
0.10 ms	0	



Pengujian 4: google search maxw1n (untuk uji weighted-levenstein, pattern matching dan tooltip)

Google

maxwin

Instagram

polaaaaarandom

24.2 rb+ pengikut

POLA **MAXWIN** MAHJONG (@polaaaaarandom)

24K followers · 426 following · 67 posts · @polaaaaarandom: "" Dilarang berhenti sebelum modal kembali " Berani mencoba? Follow @**random77_official**"

kyuubikurama shop

https://kyuubikurama.shop

AsupanToto - Penuhi Kebutuhan **Maxwin** Harian Anda ...

10 Last Deposit · 10 Last Withdraw Selengkapnya. Username, Withdraw. huj****20, Rp 5.500.000. nev*****2, Rp 8.400 ...

YouTube

https://www.youtube.com/...

Terjemahkan halaman ini

MaxWin

Share your videos with friends, family, and the world.

Linktree

https://linktr.ee/ **MAXWIN**

MAXWIN | Instagram, Facebook

GRATIS SPIN HINGGA 200x. LINK DAFTAR. Instagram, FACEBOOK. Explore other Linktrees.

jumperspodcast@jumperspodcast. Alyson Stoner@alysonstoner.

Frekuensi: 20 kali

Total Waktu Eksekusi: 1975.800 ms

maxx - Weighted-Levenshtein: 1975.800 ms

maxxwin1.com

https://maxxwin1.com

Terjemahkan halaman ini

Maxxwin | #1 Sports Betting & Casino Gaming Platform

Maxxwin is an online casino and **sportsbook** where players can enjoy a wide range of **slots**, **jackpot** games, **live casino** tables and sports betting in one place.

Instagram

pecandu_wedde

10+ saku · 1 tahun yang lalu

Pola dari kk @media.**maxwin** blm berhasil ya kk 🍀 Yuk ...

Pola dari kk @media.**maxwin** blm berhasil ya kk 🍀 Yuk guys! Btwin challenge terbaru dari aku ... Link

linktr.ee/MAXWIN

SWEETBONANZA

Judol Detector

Statistik scan terbaru diterima.

Total Keyword Ditemukan

32

Scan Terakhir

HALAMAN
maxw1n - Penelusuran Google

DOMAIN
www.google.com

WAKTU
30 Mei 2026, 20.55

Statistik Algoritma

Boyer-Moore

8 match

WAKTU
47.20 ms

COMPARISON
207133

KMP

8 match

WAKTU

COMPARISON

KMP

8 match

WAKTU
68.00 ms

COMPARISON
6232605

Regex

2 match

WAKTU
1.90 ms

COMPARISON
0

Weighted-Levenshtein

22 match

WAKTU
2118.40 ms

COMPARISON
0

Aho-Corasick

8 match

WAKTU
213.40 ms

COMPARISON
8979

Rabin-Karp

8 match

WAKTU
63.90 ms

COMPARISON
13912

OCR

0 match

WAKTU
0.00 ms

INPUT
0

gle

maxwin

Instagram

polaaaaarandom

24.2 rb+ pengikut

POLA **MAXWIN** MAHJONG (@polaaaaarandom)

24K followers · 426 following · 67 posts · @polaaaaarandom: "" Dilarang berhenti sebelum modal kembali " Berani mencoba? Follow @**random77_official**"

kyuubikurama shop

https://kyuubikurama.shop

AsupanToto - Penuhi Kebutuhan **Maxwin** Harian Anda ...

10 Last Deposit · 10 Last Withdraw Selengkapnya. Username, Withdraw. huj****20, Rp 5.500.000. nev*****2, Rp 8.400 ...

YouTube

https://www.youtube.com/...

Terjemahkan halaman ini

MaxWin

Share your videos with friends, family, and the world.

Linktree

https://linktr.ee/ **MAXWIN**

MAXWIN | Instagram, Facebook

GRATIS SPIN HINGGA 200x. LINK DAFTAR. Instagram, FACEBOOK. Explore other Linktrees.

jumperspodcast@jumperspodcast. Alyson Stoner@alysonstoner.

Frekuensi: 20 kali

Total Waktu Eksekusi: 1975.800 ms

maxx - Weighted-Levenshtein: 1975.800 ms

maxxwin1.com

https://maxxwin1.com

Terjemahkan halaman ini

Maxxwin | #1 Sports Betting & Casino Gaming Platform

Maxxwin is an online casino and **sportsbook** where players can enjoy a wide range of **slots**, **jackpot** games, **live casino** tables and sports betting in one place.

Instagram

pecandu_wedde

10+ saku · 1 tahun yang lalu

Pola dari kk @media.**maxwin** blm berhasil ya kk 🍀 Yuk ...

Pola dari kk @media.**maxwin** blm berhasil ya kk 🍀 Yuk guys! Btwin challenge terbaru dari aku ... Link

linktr.ee/MAXWIN

SWEETBONANZA

Judol Detector

Statistik scan terbaru diterima.

Total Keyword Ditemukan

32

Scan Terakhir

HALAMAN
maxw1n - Penelusuran Google

DOMAIN
www.google.com

WAKTU
30 Mei 2026, 20.55

Statistik Algoritma

Boyer-Moore

8 match

WAKTU
47.20 ms

COMPARISON
207133

KMP

8 match

WAKTU

COMPARISON

KMP

8 match

WAKTU
68.00 ms

COMPARISON
6232605

Regex

2 match

WAKTU
1.90 ms

COMPARISON
0

Weighted-Levenshtein

22 match

WAKTU
2118.40 ms

COMPARISON
0

Aho-Corasick

8 match

WAKTU
213.40 ms

COMPARISON
8979

Rabin-Karp

8 match

WAKTU
63.90 ms

COMPARISON
13912

OCR

0 match

WAKTU
0.00 ms

INPUT
0

50

Google

maxwin

POLA **MAXWIN** MAHJONG (@polaaaaarandom)

24K followers · 426 following · 67 posts · @polaaaaarandom: "" Dilarang berhenti sebelum modal kembali " Berani mencoba? Follow @random77 _official"

kyuubikurama.shop

https://kyuubikurama.shop

AsupanToto - Penuhi Kebutuhan **Maxwin** Harian Anda ...

10 Last Deposit · 10 Last Withdraw Selengkapnya. Username, Withdraw. hu*****20, Rp 5.500.000. ney*****2, Rp 8.400 ...

YouTube

https://www.youtube.com/... · ... · Terjemahkan halaman ini

MaxWin

Share your videos with friends, family, and the world.

Linktree

https://lnkd.in.g/... · ... · Terjemahkan halaman ini

MAXWIN | Instagram, Facebook

GRATIS SPIN HINGGA 200x. LINK DAFTAR: Instagram. FACEBOOK. Explore other Linktrees. jumperspodcast | jumperspodcast. Alyson Stoner | alysonstoner.

maxwin1.com

https://maxwin1.com · ... · Terjemahkan halaman ini

Maxwin | #1 Sports Betting & Casino Gaming Platform

Maxwin is an online casino and **sportsbook** where players can enjoy a wide range of **slots**, **jackpot** games, **live casino** tables and sports betting.

Keyword: sportsbook

Algoritma: Boyer-Moore, KMP, Aho-Corasick, Rabin-Karp

Tekst yang dicari: 2 kali

Total Waktu Eksekusi:

- Boyer-Moore: 54.300 ms

- KMP: 72.700 ms

- Aho-Corasick: 225.900 ms

- Rabin-Karp: 69.300 ms

Instagram: peccandu.wedde

1b+ foto · 1 tahun yang lalu

Pola dari kk @media.**maxwin** t

Pola dari kk @media **maxwin** bin berhasil

main diblo ✓ ... #leads #eadisensasional #

Rabin-Karp

9 match

WAKTU

69.30 ms

COMPARISON

14086

OCR

0 match

WAKTU

0.00 ms

INPUT

0

Perbandingan Keyword

maxwin 19

slot 3

maxwin 2

Rescan Halaman

☐ OCR gambar

OCR nonaktif.

☐ Blur teks

Blur teks nonaktif.

Pengujian 5: google search gacor999 (untuk blur teks)

Google

gacor999

Mode AI

Semua

Video

Berita

Gambar

Shopping

Video singkat

Lainnya

Alat

newhamtaxiscabs.co.uk

https://www.newhamtaxiscabs.co.uk/... · id:GACOR999+...

GACOR999 LOGIN : Layanan Taksi 24 Jam Newham untuk ...

GACOR999 LOGIN menyediakan layanan transportasi profesional 24 jam untuk perjalanan lokal maupun jarak jauh di London. USS\$95,00

Gambar

SLOT AXWIN

RAJA RULANG & RYAN RULANG ITUSAN JUTA!

DAFTAR

GACOR999

King Invitation

GACOR999

7ms garutkab.go.id

GACOR999

E Office Kampar ...

GACOR999

e-office Sumedang - Kam...

Tampilkan gambar lainnya

Ringkasan AI

Istilah **Gacor999** umumnya merujuk pada platform atau situs yang berkaitan dengan permainan slot online. Namun, perlu diketahui bahwa sebagian besar situs dengan nama semacam ini beroperasi secara ilegal dan tidak memiliki lisensi resmi dari otoritas

Rabin-Karp

59 match

WAKTU

64.50 ms

COMPARISON

15282

OCR

0 match

WAKTU

0.00 ms

INPUT

0

Perbandingan Keyword

gacor 43

slot 6

lotto12 4

Rescan Halaman

☐ OCR gambar

OCR nonaktif.

☐ Blur teks

Blur teks nonaktif.

gacor999

Mode AI Semua Video Berita Gambar Shopping Video singkat Lainnya Alat

newhamtaxiscabs.co.uk

https://www.newhamtaxiscabs.co.uk/id-999+...

999 LOGIN : Layanan Taksi 24 Jam Newham untuk ...

999 LOGIN menyediakan layanan transportasi profesional 24 jam untuk perjalanan lokal maupun jarak jauh di London.

US\$969,00

Gambar

999: 999 Raja...

King Invitation

999

rms.garutkab.go.id

999 : E Office Kampar ...

e-office Sumedang - Kam...

Tampilkan gambar lainnya

Ringkasan AI

Istilah 999 umumnya merujuk pada platform atau situs yang berkaitan dengan permainan slot online. Namun, perlu diketahui bahwa sebagian besar situs dengan nama semacam ini beroperasi secara ilegal dan tidak memiliki lisensi resmi dari otoritas perjudian yang sah.

Rabin-Karp

59 match

WAKTU

64.50 ms

COMPARISON

15282

OCR

0 match

WAKTU

0.00 ms

INPUT

0

Perbandingan Keyword

king999

2

jackpot

2

fauntrah25

2

Rescan Halaman

OCR gambar

OCR nonaktif.

Blur teks

Blur teks aktif untuk hasil deteksi.

4.5.2. Hasil Performa Algoritma

Kasus Uji	Halaman / Query	Algoritma	Jml Match	Jml Perbandingan	Waktu Eksekusi (ms)
PENGUJIAN 1 — Google Search gacor					
Data Besar	google.com, gacor	Boyer-Moore	155	322751	92,2
Data Besar	google.com, gacor	KMP	155	8065442	84,7
Data Besar	google.com, gacor	Regex	11	0	2,2
Data Besar	google.com, gacor	Weighted-Levenshtein	1	0	2792,6
Data Besar	google.com, gacor	Aho-Corasick	155	12289	268,3
Data Besar	google.com, gacor	Rabin-Karp	155	23199	95,5
Data Besar	google.com, gacor	OCR	0	1	0
PENGUJIAN 2 — Google Search merdeka 45					
Data Kecil	google.com, merdeka 45	Boyer-Moore	0	416209	68
Data Kecil	google.com, merdeka 45	KMP	0	10093092	95
Data Kecil	google.com, merdeka 45	Regex	17	0	2,6

Data Kecil	google.com, merdeka 45	Weighted-Levenshtein	0	0	3572
Data Kecil	google.com, merdeka 45	Aho-Corasick	0	16321	264
Data Kecil	google.com, merdeka 45	Rabin-Karp	0	30878	122,4
Data Kecil	google.com, merdeka 45	OCR	0	6	8502
PENGUJIAN 3 — hantu-unjukgigi.pages.dev					
Data Sedang	hantu-unjukgigi.page s.dev	Boyer-Moore	1	3807	2,2
Data Sedang	hantu-unjukgigi.page s.dev	KMP	1	138302	2,4
Data Sedang	hantu-unjukgigi.page s.dev	Regex	0	0	0,1
Data Sedang	hantu-unjukgigi.page s.dev	Weighted-Levenshtein	0	0	47
Data Sedang	hantu-unjukgigi.page s.dev	Aho-Corasick	1	179	7,7
Data Sedang	hantu-unjukgigi.page s.dev	Rabin-Karp	1	202	1,8
Data Sedang	hantu-unjukgigi.page s.dev	OCR (tanpa aktif)	0	0	0
Data Sedang	hantu-unjukgigi.page s.dev	OCR (aktif)	1	4	2162,5
PENGUJIAN 4 — Google Search maxw1n					
Data Sedang	google.com, maxw1n	Boyer-Moore	8	207133	47,2
Data Sedang	google.com, maxw1n	KMP	8	6232605	68
Data Sedang	google.com, maxw1n	Regex	2	0	1,9
Data Sedang	google.com, maxw1n	Weighted-Levenshtein	22	0	2118,4
Data Sedang	google.com, maxw1n	Aho-Corasick	8	8979	213,4
Data Sedang	google.com, maxw1n	Rabin-Karp	8	13912	63,9
Data Sedang	google.com, maxw1n	OCR	0	0	0

PENGUJIAN 5 — Google Search gacor999

Data Besar	google.com, gacor999	Boyer-Moore	59	15282	64,5
Data Besar	google.com, gacor999	KMP	59	15282	64,5
Data Besar	google.com, gacor999	Rabin-Karp	59	15282	64,5
Data Besar	google.com, gacor999	OCR	0	0	0

4.5.3. Analisis Hasil Pengujian**1. Analisis Efisiensi Perbandingan Karakter (Character Comparisons)**

Berdasarkan data dari kelima pengujian yang dilakukan, terlihat perbedaan yang sangat mencolok dalam jumlah operasi perbandingan karakter (character comparisons) yang dilakukan oleh masing-masing algoritma. Algoritma Boyer-Moore secara konsisten menunjukkan efisiensi perbandingan yang jauh lebih tinggi dibandingkan Knuth-Morris-Pratt (KMP). Sebagai contoh, pada Pengujian 1 (Google Search gacor dengan 106 keyword), Boyer-Moore hanya melakukan 322.751 kali perbandingan karakter, sedangkan KMP melonjak drastis hingga mencapai 8.065.442 kali perbandingan. Hal ini membuktikan efektivitas heuristik karakter buruk (*Bad Character Heuristic*) pada Boyer-Moore yang melakukan pencocokan dari kanan ke kiri, sehingga mampu melakukan lompatan besar (*skipping shifts*) melewati puluhan karakter teks yang tidak relevan tanpa perlu membandingkannya satu per satu.

Di sisi lain, algoritma bonus Aho-Corasick mencatatkan jumlah perbandingan yang paling minimal di antara seluruh metode pencarian eksak, yaitu hanya sebesar 12.289 perbandingan pada Pengujian 1. Hasil ini diperoleh karena Aho-Corasick mengompilasi seluruh kata kunci dari berkas keywords.txt ke dalam satu pohon otomatis (Trie Automaton) pada fase pra-pemrosesan. Akibatnya, Aho-Corasick dapat memeriksa semua kata kunci secara sekaligus dalam satu kali pemindaian teks dari kiri ke kanan (single pass), meminimalkan redundansi secara optimal. Sementara itu, Rabin-Karp menunjukkan jumlah perbandingan tingkat menengah (misalnya 23.199 pada Pengujian 1) karena ia membandingkan nilai numerik hash terlebih dahulu dan hanya melakukan perbandingan karakter secara manual apabila terdeteksi kecocokan hash (kolisi). Hal ini membuktikan bahwa pencarian berbasis hash sukses mereduksi perbandingan biner karakter secara signifikan di memori.

2. Analisis Kecepatan Eksekusi

Meskipun jumlah perbandingan karakter KMP jauh lebih banyak daripada Boyer-Moore, waktu eksekusi aktualnya menunjukkan fenomena yang menarik. Pada Pengujian 1, KMP selesai dalam waktu 84.7 ms, sedikit lebih cepat

daripada Boyer-Moore yang membutuhkan waktu 92.2 ms. Fenomena ini terjadi karena KMP memiliki struktur loop yang jauh lebih sederhana tanpa melibatkan pencarian peta dinamis (*dynamic Map lookup*) seperti *Last Occurrence Table* pada Boyer-Moore. Oleh karena itu, dengan mudah mengoptimalkan operasi primitif KMP menjadi instruksi bahasa mesin yang berjalan sangat cepat, mengimbangi beban perbandingan karakternya yang tinggi.

Sebaliknya, algoritma Aho-Corasick yang memiliki jumlah perbandingan karakter paling sedikit, justru mencatatkan waktu eksekusi yang lebih lambat, yaitu 268.3 ms pada Pengujian 1. Hal ini disebabkan oleh besarnya beban pra pemrosesan (*preprocessing overhead*) untuk menginstansiasi objek-objek simpul Trie, tabel transisi anak, dan pembuatan tautan kegagalan (*Failure Link*) tingkat demi tingkat menggunakan metode BFS Queue di dalam memori Heap JavaScript. Hal ini mempertegas bahwa Aho-Corasick baru akan menunjukkan keunggulan kecepatan mutlak apabila struktur Trie-nya dibangun sekali saja di awal aplikasi, bukan diinstansiasi berulang kali di setiap proses pemindaian.

Di sisi lain, algoritma Weighted Levenshtein Distance dan Optical Character Recognition (OCR) mencatatkan waktu eksekusi yang paling lambat di antara semuanya. Weighted Levenshtein Distance membutuhkan waktu sekitar 2.100 ms hingga 3.500 ms karena harus membangun matriks pemrograman dinamis dua dimensi $O(N \times M)$ untuk membandingkan setiap token kandidat di halaman web terhadap daftar kata kunci. Kebutuhan komputasi dan alokasi memori ini sangat berat sehingga hanya cocok digunakan sebagai penyaring khusus teks termanipulasi. Sementara itu, fitur OCR (Tesseract.js) menjadi proses terlama dengan waktu rata-rata di atas 2.000 ms hingga mencapai 9.800 ms per pengujian, karena harus melakukan prapemrosesan piksel gambar secara visual, memuat model jaringan saraf (neural network), dan menjalankan ekstraksi karakter asinkron di dalam Web Worker latar belakang.

3. Analisis Sensitivitas Deteksi dan Akurasi Pencocokan (Exact vs. Fuzzy vs. RegEx)

Dari sisi fungsionalitas dan sensitivitas deteksi, kelima pengujian tersebut memperlihatkan bagaimana pembagian peran dalam hierarki pencocokan berjalan secara optimal. Algoritma pencarian eksak (KMP, Boyer-Moore, Aho-Corasick, dan Rabin-Karp) hanya mampu menangkap kata kunci yang tertulis secara persis tanpa modifikasi. Ketika dihadapkan pada kata kunci yang disamarkan menggunakan leet-speak pada Pengujian 4 (Google Search maxw1n), seluruh algoritma eksak ini mengalami kebuntuan dan hanya mencatatkan 8 kecocokan eksak biasa. Sebaliknya, Weighted Levenshtein Distance menjadi satu-satunya algoritma yang sukses mendeteksi penyamaran visual tersebut dengan mencatatkan jumlah kecocokan terbanyak, yaitu 22 match (berhasil memetakan token manipulatif maxw1n menjadi kata kunci asli maxwin karena biaya substitusi karakter serupa $i \leftrightarrow 1$ bernilai sangat murah, yaitu 0.2).

Sementara itu, Regular Expression (RegEx) membuktikan efektivitasnya dalam menangkap pola dinamis berbasis pola struktural <kata><angka>. Pada Pengujian 2 (Google Search merdeka 45), ketika semua algoritma pencarian kata kunci eksak dan fuzzy mencatatkan hasil nihil (0 match), RegEx sukses mendeteksi 17 match karena berhasil mengidentifikasi kata-kata yang diikuti angka secara langsung (seperti merdeka45). Hal ini menegaskan bahwa kombinasi RegEx dan Weighted Levenshtein bertindak sebagai baris pertahanan kokoh untuk mengatasi teknik penyamaran promosi judi online yang dinamis, sedangkan metode eksak bertindak sebagai penyaring cepat untuk kata kunci standar.

5. Kesimpulan dan Saran

5.1. Kesimpulan

Pada tugas besar ini, kami berhasil mengimplementasikan SweetBonanza Judol Detector, yaitu Chromium browser extension berbasis TypeScript yang dapat mendeteksi konten berindikasi judi online pada halaman web. Sistem melakukan pemindaian terhadap text node pada DOM dan membaca daftar kata kunci dari berkas keywords.txt. Teks yang terdeteksi diberi highlight, dapat disensor menggunakan efek blur, serta dilengkapi dengan tooltip yang menampilkan informasi hasil pencocokan. Statistik pemindaian juga ditampilkan secara real-time melalui popup extension.

Pendeteksian teks dilakukan menggunakan beberapa metode yang saling melengkapi. Knuth-Morris-Pratt dan Boyer-Moore digunakan untuk mencari kata kunci secara eksak, sedangkan Regular Expression digunakan untuk mengenali pola gabungan kata dan angka. Weighted Levenshtein Distance diterapkan untuk mendeteksi variasi kata yang disamarkan melalui substitusi karakter yang serupa secara visual. Selain itu, algoritma bonus Aho-Corasick dan Rabin-Karp berhasil diimplementasikan sebagai alternatif pencarian pola. Fitur OCR berbasis Tesseract.js juga memungkinkan sistem mendeteksi teks yang disisipkan pada gambar dan menyensor gambar yang teridentifikasi.

Hasil pengujian menunjukkan bahwa setiap algoritma memiliki karakteristik yang berbeda. Boyer-Moore cenderung membutuhkan lebih sedikit perbandingan karakter daripada KMP karena dapat melakukan pergeseran pola secara lebih jauh ketika terjadi ketidakcocokan. Aho-Corasick mencatat jumlah perbandingan yang rendah karena mampu memproses banyak kata kunci dalam satu kali pemindaian teks, meskipun pembangunan struktur trie menimbulkan biaya awal tambahan. Weighted Levenshtein Distance dan OCR membutuhkan waktu eksekusi lebih besar, tetapi keduanya memperluas kemampuan deteksi terhadap teks

termanipulasi dan teks pada gambar yang tidak dapat ditangani oleh pencocokan eksak.

Secara keseluruhan, extension yang dikembangkan telah memenuhi kebutuhan utama sistem deteksi konten judi online pada halaman web serta dilengkapi dengan fitur bonus untuk meningkatkan cakupan deteksi dan penyensoran visual. Kombinasi beberapa algoritma memungkinkan sistem menangani bentuk konten yang lebih beragam dibandingkan penggunaan satu metode pencocokan saja.

5.2. Saran

Berdasarkan hasil implementasi dan pengujian, terdapat beberapa pengembangan yang dapat dilakukan untuk meningkatkan kualitas SweetBonanza Judol Detector.

1. Struktur *trie* dan *failure link* pada Aho-Corasick sebaiknya dibangun satu kali ketika daftar keyword dimuat, kemudian digunakan kembali pada proses pemindaian berikutnya. Pendekatan ini dapat mengurangi biaya *preprocessing* yang menyebabkan waktu eksekusi Aho-Corasick masih cukup tinggi.
2. Weighted Levenshtein Distance perlu dioptimalkan dengan memperketat penyaringan token kandidat atau menjalankannya hanya ketika hasil *exact matching* tidak ditemukan. Hal ini dapat mengurangi jumlah perbandingan yang tidak diperlukan karena fuzzy matching memiliki biaya komputasi yang relatif besar.
3. Pemrosesan OCR dapat ditingkatkan melalui mekanisme *caching*, pembatasan jumlah gambar, serta pemindaian berdasarkan prioritas. OCR sebaiknya dijalankan hanya pada gambar baru atau gambar yang mengalami perubahan agar waktu pemrosesan lebih efisien.
4. Pola RegEx dapat dikembangkan dengan menambahkan validasi kontekstual atau daftar kata dasar yang relevan. Langkah ini diperlukan untuk mengurangi kemungkinan *false positive* pada teks umum yang memiliki format gabungan huruf dan angka.
5. Pengujian lanjutan sebaiknya dilakukan menggunakan dataset yang lebih beragam, mencakup teks normal, teks termanipulasi, halaman dinamis, dan gambar dengan kualitas yang berbeda. Hasil pengujian tersebut dapat digunakan untuk menentukan nilai threshold fuzzy matching yang lebih optimal serta mengukur tingkat akurasi sistem secara lebih sistematis

LAMPIRAN

Link Repository GitHub

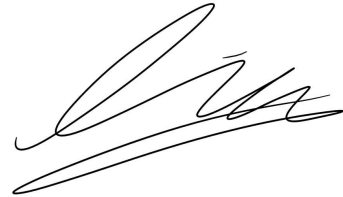
https://github.com/bangedaw/Tubes3_SweetBonanza.git

Tabel Checklist

No	Poin	Ya	Tidak
1	Extension berhasil di-build dan di-load tanpa kesalahan pada chromium browser dan dikembangkan dengan TypeScript	✓	
2	KMP dan Boyer-Moore diimplementasikan from scratch	✓	
3	Regex handle format <kata><angka> dan berbagai edge case	✓	
4	Pencarian KMP & BM membaca keyword.txt secara iteratif dan tidak menggunakan built-in search function atau library eksternal	✓	
5	Exact matching dan fuzzy matching berjalan benar	✓	
6	Elemen DOM terdeteksi diberi highlight dan terhapus saat rescanning	✓	
7	Tooltip muncul saat hover dengan informasi keyword, algoritma, kemunculan, dan waktu eksekusi	✓	
8	Popup menampilkan statistik realtime (total keyword, perbandingan, waktu eksekusi, jumlah match)	✓	
9	[Bonus] Membuat Video		✓
10	[Bonus] Implementasi Algoritma Aho-Corasick dan Rabin Karp	✓	
11	[Bonus] Implementasi Censorship / Blur Teks	✓	
12	[Bonus] Implementasi Optical Character Recognition pada Gambar	✓	

Pernyataan Tidak Melakukan Kecurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Edward David Rumahorbo



Nathanael Gunawan



Manuel Thimoty Silalahi

Tabel Pembagian Tugas

NIM	Nama	Tanggung Jawab
13524036	Edward David Rumahorbo	Implementasi OCR, Popup dan Statistik, Dokumentasi pada README
13524066	Nathanael Gunawan	Implementasi algoritma KMP, Boyer-Moore, Regex, Weighted Levenshtein Distance, algoritma bonus

13524102	Manuel Thimoty Silalahi	Implementasi manifest, DOM Walker, integrasi Algoritma, dan implementasi Algoritma bonus.
----------	-------------------------	---

Referensi

- Chrome Extension Documentation
<https://developer.chrome.com/docs/extensions/>
- Typescript
<https://www.w3schools.com/typescript/>
- Tesseract.js
<https://tesseract.projectnaptha.com/>
- Boyer Moore
<https://www.geeksforgeeks.org/dsa/boyer-moore-algorithm-for-pattern-searching/>
- KMP
<https://www.geeksforgeeks.org/dsa/kmp-algorithm-for-pattern-searching/>
- Regex
<https://www.statsig.com/perspectives/what-is-regex-a-beginner's-guide-to-pattern-matching>
- Levenshtein Distance
<https://devopedia.org/levenshtein-distance>
- Aho-Corasick
<https://www.geeksforgeeks.org/dsa/aho-corasick-algorithm-pattern-searching/>
- Rabin Karp
<https://www.geeksforgeeks.org/dsa/rabin-karp-algorithm-for-pattern-searching/>
- Materi-Materi Perkuliahan
<https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/stima25-26.htm>